Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mstr functions

Synopsis

The functions prefixed with Mstr make up the String Reader module. The String Reader module performs character recognition using feature-based technology. The module is invariant to changes in scale, aspect ratio, and contrast. String Reader also offers considerable tolerance towards variations in perspective and angle. In addition, String Reader supports multiple user-defined grammar rules and multi-font definitions in a single context, and features a powerful Graphical User Interface (GUI) for easy font definition. String Reader also offers an expert function, allowing you to quickly determine why your application is not reading the strings that you expect it to read. These features make the String Reader module the ideal tool for writing seemingly complex applications, such as Automatic Number Plate Recognition (ANPR) programs.

Functions

MstrAlloc
MstrAllocResult
MstrControl
MstrDraw
MstrEditFont
MstrExpert
MstrFree
MstrGetResult
MstrInquire
MstrPreprocess
MstrRead
MstrRestore
MstrSave
MstrSetConstraint
MstrStream

MstrAlloc

Synopsis

Allocate a String Reader context.

Syntax

```
AIL_ID MstrAlloc(
    AIL_ID SysId,          //in
    AIL_INT64 ContextType, //in
    AIL_INT64 ControlFlag, //in
    AIL_ID *ObjectIdPtr    //out
)
```

Description

This function allocates a String Reader context on the specified system. A String Reader context contains the read parameters, the list of fonts, and the list of string models.

> Note that you cannot allocate a fontless context. You must restore one of the predefined fontless context that is distributed with Aurora Imaging Library using MstrRestore().

When the String Reader context is no longer required, release it using MstrFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the String Reader context's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the String Reader context.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextType

Specifies the type of String Reader context. This parameter must be set to the value below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the context																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FONT_BASED	Specifies a font-based context.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ObjectIdPtr

Specifies the address of the variable in which to write the String Reader context identifier or specifies the data type that the function should use to return the String Reader context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated String Reader context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated String Reader context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_STR_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the String Reader context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated String Reader context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the String Reader context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_STR_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib; ailstr11.lib.
DLL	Requires ail11.dll; ailstr11.dll.

MstrAllocResult

Synopsis

Allocate a String Reader result buffer.

Syntax

```
AIL_ID MstrAllocResult(
    AIL_ID SysId,          //in
    AIL_INT64 ControlFlag, //in
    AIL_ID *ObjectIdPtr    //out
)
```

Description

This function allocates a String Reader result buffer, on the specified system, to store results obtained from an MstrRead() operation.

When the String Reader result buffer is no longer required, release it using MstrFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the String Reader result buffer's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the String Reader result buffer.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ObjectIdPtr

Specifies the address of the variable in which to write the String Reader result buffer identifier or specifies the data type that the function should use to return the String Reader result buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated String Reader result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated String Reader result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_STR_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the String Reader result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated String Reader result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the String Reader result buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_STR_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailstr11.lib.
DLL	Requires ail11.dll; ailstr11.dll.

MstrControl

Synopsis

Control a String Reader context, result buffer, a specific string model, or a specific font.

Syntax

```
void MstrControl(
    AIL_ID ContextOrResultId, //out
    AIL_INT Index,            //in
    AIL_INT64 ControlType,    //in
    AIL_DOUBLE ControlValue   //in
)
```

Description

This function sets the specified control for a String Reader context, for one (or all) of the specified string models or fonts contained therein, or a result buffer. These settings control the execution of MstrRead() operations. All the control type settings can be inquired using MstrInquire().

Changing certain control type settings might require preprocessing the String Reader context again, using MstrPreprocess(). To know if the String Reader context needs to be preprocessed, call MstrInquire() with M_PREPROCESSED.

Unless otherwise specified, the control type settings are applicable to both font-based and fontless contexts.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MstrControl()** internally calls the *AIL_DOUBLE* version of this function (MstrControlDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MstrControlInt64()).

Parameters

ContextOrResultId

Specifies the String Reader context or result buffer whose settings to modify. The String Reader context must have been previously allocated on the system using MstrAlloc(). The String Reader result buffer must have been previously allocated on the system using MstrAllocResult().

Index

Specifies that the String Reader context, a specific font, a specific string model, or a result buffer is controlled. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a String Reader context, specific font, specific string model, or result buffer															
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default. If a String Reader context is specified, same as M_CONTEXT. If a String Reader result buffer is specified, same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_INDEX(AIL_INT FontIndex)	Specifies to apply the control settings to a font. Note that this value is only available for a font-based context.	c		e	f	g	a		i		j	k	l	m	n	b

		Parameters													
		FontIndex This parameter specifies the index of the font. You can set this parameter to one of the following:													
		M_ALL Specifies to apply the control settings to all fonts.	c		e	f	g	a		i		j	k	l	m	n	b
		0 <= *Value* < M_NUMBER_OF_FONTS Specifies the index of the individual font to which to apply the control settings. User labels cannot be used as indices; to retrieve the index of a font from its user label, use MstrInquire() with M_FONT_INDEX_FROM_LABEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_INDEX(AIL_INT StringIndex)		Specifies to apply the control settings to a string.	c		e	f	g	a		i		j	k	l	m	n	b
		Parameters															
		StringIndex This parameter specifies the index of the string. You can set this parameter to one of the following:															
		M_ALL Specifies to apply the control settings to all strings. Note that to apply control settings to all strings, the ControlType and ControlValue parameters must be supported by all the string models.	c		e	f	g	a		i		j	k	l	m	n	b
		0 <= *Value* < M_NUMBER_OF_STRING_MODELS Specifies the index of the individual string to which to apply the control settings. User labels cannot be used as indices; to retrieve the index of a string from its user label, use MstrInquire() with M_STRING_INDEX_FROM_LABEL.	c		e	f	g	a		i		j	k	l	m	n	b

M_CONTEXT	Specifies that a general setting of a String Reader context will be controlled.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies that a general setting of a String Reader result buffer will be controlled, if one is specified.	c		e	f	g	a		i		j	k	l	m	n	b

ControlType

Specifies the setting to change.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the setting's new value.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- For the context or a specific string
- For the context
- For a specific string
- For a specific font
- For a font-based or a fontless context
- For the result buffer

The following ControlType and corresponding ControlValue parameter settings can be specified for either the String Reader context (M_CONTEXT) or each specific string in the String Reader context (M_STRING_INDEX()).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the context or a specific string																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_STRING_NUMBER		Sets either the maximum (total) number of all strings that can be read with a String Reader context (M_CONTEXT), or the maximum number of a specific string (M_STRING_INDEX()) that can be read with a String Reader context. INQ															
	M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies that all strings will be read. Note that this setting can increase the read time; always set M_STRING_NUMBER to a specific number whenever possible.	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the maximum number of strings that can be read. For M_CONTEXT, this value specifies the maximum (total) number of strings that can be read. Any value greater than or equal to 1 can be set. For M_STRING_INDEX(), this value specifies the maximum number of a specific string model that can be read. Any value greater than or equal to 0 can be set.	c		e	f	g	a		i		j	k	l	m	n	b

The possible ControlType and corresponding ControlValue parameter settings for the String Reader context (M_CONTEXT) are:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the context																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_DISABLE_CHAR		Disables the specified character in the fontless context. Only uppercase characters and numbers can be disabled. The String Reader module will not try to match found characters with disabled characters. Note that this control type is only available for a fontless context. INQ															
	M_ALL	Specifies that no characters will be read.	c		e	f	g	a		i		j	k	l	m	n	b
	ASCII value	Specifies the ASCII value of the character to disable.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE_CHAR		Enables the specified character in the fontless context. Only uppercase characters and numbers can be enabled. The String Reader module will only try to identify enabled characters. Note that this control type is only available for a fontless context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies that all characters will be read. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	ASCII value	Specifies the ASCII value of the character.	c		e	f	g	a		i		j	k	l	m	n	b

M_ENCODING	Sets the type of character encoding used by the String Reader context. All functions that accept or return a string of characters as a parameter use character encoding to determine the type of the pointer. The type of character encoding is also used as the default result type (MstrGetResult()) for a string of characters. The character encoding cannot be changed when there are characters in the context whose ASCII character values lie outside the range [0, 127]. This applies to characters which are in fonts and constraints. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ASCII.	c		e	f	g	a		i		j	k	l	m	n	b
M_ASCII	Specifies an 8-bit ASCII standard. This corresponds to the *char* data type.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNICODE	Specifies a 16-bit Unicode standard. This corresponds to the *short* data type.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_ADD	Adds an empty font to the String Reader context. The maximum number of fonts in a String Reader context is 255. Note that this control type is only available for a font-based context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_USER_DEFINED.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies a user-defined type of font. Initially, the font is empty. To add characters to the font, or to edit it, use MstrEditFont().	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_DELETE	Deletes a font from the String Reader context. Note that this control type is only available for a font-based context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies that all fonts will be deleted.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the font to delete.	c		e	f	g	a		i		j	k	l	m	n	b

M_MAX_CHAR_SIZE_X	Sets the width (X-size) of the widest enabled character in the fontless context. Specify the width that the character has when it has a height of M_REF_CHAR_SIZE_Y. If the read algorithm encounters a character with a height other than M_REF_CHAR_SIZE_Y, the maximum width will be interpolated. Note that this control type is only available for a fontless context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 65 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 65536	Specifies the maximum character width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_CHAR_SIZE_X	Sets the width (X-size) of the narrowest enabled character in the fontless context. Specify the width that the character has when it has a height of M_REF_CHAR_SIZE_Y. If the read algorithm encounters a character with a height other than M_REF_CHAR_SIZE_Y, the minimum width will be interpolated. Note that this control type is only available for a fontless context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 6 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 65536	Specifies the minimum character width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_MINIMUM_CONTRAST	Sets the minimum contrast between a character in the target image and its background, in order for the character to be read by MstrRead(). Increasing the minimum contrast will typically speed up the read operation, particularly in presence of noise; however, if the minimum contrast is higher than the contrast of an actual character, that character will be lost. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 15.	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* <= 255	Specifies the minimum contrast.	c		e	f	g	a		i		j	k	l	m	n	b

M_REF_CHAR_SIZE_Y	Sets the height which will be the reference for M_MIN_CHAR_SIZE_X, M_MAX_CHAR_SIZE_X, and M_REF_CHAR_THICKNESS. The reference height is the height at which you specify the width of the narrowest and widest characters, as well as their thickness, with M_MIN_CHAR_SIZE_X, M_MAX_CHAR_SIZE_X, and M_REF_CHAR_THICKNESS, respectively. Note that this control type is only available for a fontless context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 46 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 65536	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_REF_CHAR_THICKNESS	Sets the thickness (stroke width) of enabled characters in the fontless context. Specify the thickness of characters when they have a height of M_REF_CHAR_SIZE_Y. If the read algorithm encounters a character with a height other than M_REF_CHAR_SIZE_Y, the maximum width will be interpolated. Note that this control type is only available for a fontless context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 6 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 65536	Specifies the thickness, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_CHAR_ANGLE	Sets whether to perform calculations specific to angular-range search strategies, for characters. The read algorithm is naturally robust to variations in a character's angle. Therefore, a character might still be read correctly, even if M_SEARCH_CHAR_ANGLE is disabled. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform calculations specific to angular-range search strategies.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform calculations specific to angular-range search strategies.	c		e	f	g	a		i		j	k	l	m	n	b

M_SEARCH_SKEW_ANGLE	Sets whether to perform calculations specific to angular-range skew search strategies, for characters. The read algorithm is naturally robust to variations in a character's skew. Therefore, a character might still be read correctly, even if M_SEARCH_SKEW_ANGLE is disabled. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform calculations specific to angular-range skew search strategies.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform calculations specific to angular-range skew search strategies.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_STRING_ANGLE	Sets whether to search for the string angle. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to search for the string angle.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to search for the string angle.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACE_CHARACTER	Sets the character value to use as a space character within the formatted string or text that will be read. M_SPACE_CHARACTER applies only when getting the formatted string (MstrGetResult() with M_FORMATTED_STRING) or the formatted text (MstrGetResult() with M_TEXT). A space character is inserted in the string each time the distance between two adjacent characters exceeds the space width, at the scale of the string. The space character must be set before the string is read. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is numerical ASCII code 32.	c		e	f	g	a		i		j	k	l	m	n	b
M_NONE	Specifies no space character.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the space character, as its numerical code (ASCII or Unicode). For example, ASCII 10 results in a new line character, ASCII 32 results in a space character. Note that the character value must be cast to an *AIL_DOUBLE*.	c		e	f	g	a		i		j	k	l	m	n	b

M_SPEED	Sets the search and read speed of the MstrRead() operation. Note that increasing the speed can decrease the robustness of MstrRead(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_MEDIUM.	c		e	f	g	a		i		j	k	l	m	n	b
M_HIGH	Specifies a high speed.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEDIUM	Specifies a medium speed.	c		e	f	g	a		i		j	k	l	m	n	b
M_STOP_EXPERT	Stops the current string expert operation. Note that you can only perform this operation from another thread of higher priority.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
M_STOP_READ	Stops the current string read operation. Note that you can only perform this operation from another thread of higher priority.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ADD	Adds a string model to the String Reader context. The maximum number of string models in a String Reader context is 255. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_USER_DEFINED.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies a user-defined type of string model. Initially, a user-defined string model does not have any constraints, does not use any checksum error detection, and all its parameters are set to their default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_DELETE	Deletes a string from the String Reader context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies that all strings will be deleted.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the string to delete.	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_SEPARATOR	Sets the character value to use as a string separator within the formatted text that will be read. M_STRING_SEPARATOR applies only when getting the formatted text (MstrGetResult() with M_TEXT). A string separator is inserted between each string in the text; that is, each time the distance between two adjacent characters is greater than the space size. The space size depends on the space width (at the scale of the string) and the maximum number of consecutive spaces. The string separator character must be set before the string is read. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is numerical ASCII code 10.	c		e	f	g	a		i		j	k	l	m	n	b
M_NONE	Specifies no separator character.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the separator character, as its numerical code (ASCII or Unicode). For example, ASCII 10 results in a new line character, ASCII 32 results in a space character. Note that the character value must be cast to an *AIL_DOUBLE*.	c		e	f	g	a		i		j	k	l	m	n	b
M_THICKEN_CHAR	Sets the number of character thickening iterations. Each iteration enlarges the thickness of the target character. This is useful for some types of fonts (for example, dotted characters). You should choose a value large enough for the intra character dots to connect but not too large to connect separate characters together. Note that the characters included in the font should not be dotted. Instead a thickened version should be included. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 100	Specifies the number of character thickening iterations.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_MODE	Sets the threshold method of the character blob extraction. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_LOCAL_WITH_RESEGMENTATION.	c		e	f	g	a		i		j	k	l	m	n	b

	M_LOCAL	Specifies a local thresholding that uses only one threshold step that is automatically computed by the module. This threshold method can be used in various contrast situations and can improve the read operation's speed, compared to M_LOCAL_WITH_RESEGMENTATION.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOCAL_WITH_RESEGMENTATION	Specifies a local thresholding that uses multiple threshold steps and split and merge techniques. This threshold method allows the module to extract character blobs in low contrast and in complex situations such as merged or broken characters.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies a global thresholding that uses the threshold value set with M_THRESHOLD_VALUE. This threshold method improves the read operation's speed, compared to M_LOCAL_WITH_RESEGMENTATION and M_LOCAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_VALUE		Sets the global threshold value when M_THRESHOLD_MODE is set to M_USER_DEFINED (otherwise it is ignored). Note that two different threshold values, which should segment the target image in exactly the same way, might produce different results, even when using a binarized target image. This is because during the read operation, the image is occasionally resized or rotated internally which can cause pixels to be interpolated. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO_COMPUTE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO_COMPUTE	Specifies to automatically compute the global threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 255	Specifies the threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT		Sets the maximum read time for MstrRead(), in msec. If the read operation times out, the strings read up to that point will be returned as a result. Note that the actual maximum read time might vary slightly from the time that you set. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 2000.0 msec.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies an infinite amount of read time.	c		e	f	g	a		i		j	k	l	m	n	b

	Value >= 0	Specifies the maximum read time, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

The possible ControlType and corresponding ControlValue parameter settings for a specific string (M_STRING_INDEX()) are:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a specific string																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_CHAR_ACCEPTANCE		Sets the acceptance level for the character score. The character's score quantifies the similarity between the character in the target and the character in the font; it also quantifies the similarity between the character in the target and the other characters of the string in the target. To be accepted as a string result, the character's score must be greater than or equal to M_CHAR_ACCEPTANCE. Scores can be retrieved with MstrGetResult(). INQ															
	M_DEFAULT	Specifies the default value; the default value is 50.0%.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies an acceptable score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_ASPECT_RATIO_MAX_FACTOR		Sets the factor used to determine the maximum acceptable aspect ratio for the characters in the string. This value is relative to the found string aspect ratio. That is, *UpperLimit* = *FoundStringAspectRatio* x M_CHAR_ASPECT_RATIO_MAX_FACTOR. This value sets the maximum possible aspect ratio variation for the characters in the string. Characters with an aspect ratio outside the aspect ratio range will not be returned as results. Note that this control type is only available for a font-based context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.1.	c		e	f	g	a		i		j	k	l	m	n	b
	1.0 <= *Value* <= 2.0	Specifies the factor that determines the maximum aspect ratio of the characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b

M_CHAR_ASPECT_RATIO_MIN_FACTOR	Sets the factor used to determine the minimum acceptable aspect ratio for the characters in the string. This value is relative to the found string aspect ratio. That is, *LowerLimit* = *FoundStringAspectRatio* x M_CHAR_ASPECT_RATIO_MIN_FACTOR. This value sets the minimum possible aspect ratio variation for the characters in the string. Characters with an aspect ratio outside the aspect ratio range will not be returned as results. Note that this control type is only available for a font-based context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.9.	c		e	f	g	a		i		j	k	l	m	n	b
0.5 <= *Value* <= 1.0	Specifies the factor that determines the minimum aspect ratio of the characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_HOMOGENEITY_ACCEPTANCE	Sets the acceptance level for the character's homogeneity score. The character's homogeneity score quantifies the similarity between the character in the target and the other characters of the string in the target. To be accepted as a string result, the character's homogeneity score must be greater than or equal to M_CHAR_HOMOGENEITY_ACCEPTANCE. Scores can be retrieved with MstrGetResult(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50.0%.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies an acceptable score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b

M_CHAR_MAX_BASELINE_DEVIATION	Sets the maximum baseline deviation that all characters defined in the font can have in the target image. A character's maximum baseline deviation is the maximum tolerable baseline deviation that a character in a string can have before it is rejected from the string. For more information on character baseline deviations, see the Character's maximum baseline deviation subsection of the Degrees of freedom section of Chapter 14: String Reader. Note that for non-punctuation characters, String Reader calculates the baseline deviation, as a percentage of each individual character's height (Y-size). For example, a baseline deviation of 10 means that a character's baseline deviates from the string's baseline by 10% of the character's height (Y-size). For punctuation characters (M_PUNCTUATION), String Reader calculates the baseline deviation as a percentage of the height of the character with the greatest Y-size in the font. For example, a baseline deviation of 10 means that the punctuation character's baseline deviates from the string's baseline by 10% of the height of the tallest character in the font. This is done because the Y-size of punctuation characters is typically too small to calculate the baseline deviation as a percentage of the character's height. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 100	Specifies the character's maximum baseline deviation within the string model.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SCALE_MAX_FACTOR	Sets the factor used to determine the upper limit (maximum permitted scale) of the scale range for the characters in the string. This value is relative to the found string scale. That is, *UpperLimit* = *FoundStringScale* x M_CHAR_SCALE_MAX_FACTOR. This value sets the possible increasing scale variation for the characters in the string. Characters with a scale outside the scale range will not be returned as results. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.1.	c		e	f	g	a		i		j	k	l	m	n	b
1.0 <= *Value* <= 2.0	Specifies the factor that determines the maximum scale of the characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b

M_CHAR_SCALE_MIN_FACTOR	Sets the factor used to determine the lower limit (minimum permitted scale) of the scale range for the characters in the string. This value is relative to the found string scale. That is, *LowerLimit* = *FoundStringScale* x M_CHAR_SCALE_MIN_FACTOR. This value sets the possible decreasing scale variation for the characters in the string. Characters with a scale outside the scale range will not be returned as results. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.9.	c		e	f	g	a		i		j	k	l	m	n	b
0.5 <= *Value* <= 1.0	Specifies the factor that determines the minimum scale of the characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SIMILARITY_ACCEPTANCE	Sets the acceptance level for the character's similarity score. The character's similarity score quantifies the similarity between the character in the target and the character in the font. To be accepted as a string result, the character's similarity score must be greater than or equal to M_CHAR_SIMILARITY_ACCEPTANCE. Scores can be retrieved with MstrGetResult(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50.0%.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies an acceptable score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_VALUE	Sets the foreground color of the string. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FOREGROUND_BLACK.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_BLACK	Specifies that black is the foreground color of the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_BLACK_OR_WHITE	Specifies that the foreground color of the string can be either black or white.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_WHITE	Specifies that white is the foreground color of the string.	c		e	f	g	a		i		j	k	l	m	n	b

M_SPACE_MAX_CONSECUTIVE	Sets the maximum number of consecutive space characters allowed in the string. Together with the space width (**MstrControl**() with M_SPACE_WIDTH), M_SPACE_MAX_CONSECUTIVE determines the maximum distance between 2 adjacent characters of the same string. If the distance between two characters is greater than the maximum distance, then they are considered to be part of 2 different strings. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 3.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number of consecutive space characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ACCEPTANCE	Sets the acceptance level for the string score. To be accepted as a string result, both the string's score and the string target score must be greater than or equal to their respective acceptance levels. The string score is the average score of the characters in the string. Scores can be retrieved with MstrGetResult(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50.0%.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies an acceptable score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE	Sets the nominal angle of the string. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ACCORDING_TO_REGION.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_REGION	Specifies to set the nominal angle to the angle of the rectangular ROI (set using MbufSetRegion()) in the image. If no region of interest has been defined, the default value is 0.0°. This implies that to read a string at an angle, the target image must have a rectangular ROI defined around the string to be read, and that ROI should be at an angle.	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_ANGLE_DELTA_NEG	Sets the lower limit of the string's angular range, relative to the nominal angle (**MstrControl**() with M_STRING_ANGLE). That is, *LowerLimit* = M_STRING_ANGLE - M_STRING_ANGLE_DELTA_NEG. This value sets the possible clockwise angular variation of the string. Strings with angles outside the angular range will not be returned as results. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 10.0	Specifies the lower limit of the angular range, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE_DELTA_POS	Sets the upper limit of the angular range, relative to the nominal angle (**MstrControl**() with M_STRING_ANGLE). That is, *UpperLimit* = M_STRING_ANGLE + M_STRING_ANGLE_DELTA_POS. This value sets the possible counter-clockwise angular variation of the string. Strings with angles outside the angular range will not be returned as results. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 10.0	Specifies the upper limit of the angular range, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ASPECT_RATIO	Sets the nominal aspect ratio of the string. The aspect ratio of the string is the median aspect ratio of all the characters in the string, while the aspect ratio of a character is the ratio of its scale in X by its scale in Y. This indicates how the width and the height of the characters in the string vary relative to the characters in the font. Note that this control type is only available for a font-based context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.5 <= *Value* <= 2.0	Specifies the ratio of pixel width to pixel height of the target.	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_ASPECT_RATIO_MAX_FACTOR	Sets the factor used to determine the upper limit of the string's aspect ratio. This value is relative to the nominal aspect ratio (**MstrControl**() with M_STRING_ASPECT_RATIO). That is, *UpperLimit* = M_STRING_ASPECT_RATIO x M_STRING_ASPECT_RATIO_MAX_FACTOR. This value sets the possible increasing aspect ratio variation for the string. Strings with an aspect ratio outside the aspect ratio range will not be returned as results. Note that this control type is only available for a font-based context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.25.	c		e	f	g	a		i		j	k	l	m	n	b
1.0 <= *Value* <= 2.0	Specifies the factor that determines the maximum aspect ratio of the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ASPECT_RATIO_MIN_FACTOR	Sets the factor used to determine the lower limit of the string's aspect ratio. This value is relative to the nominal aspect ratio (**MstrControl**() with M_STRING_ASPECT_RATIO). That is, *LowerLimit* = M_STRING_ASPECT_RATIO x M_STRING_ASPECT_RATIO_MIN_FACTOR. This value sets the possible decreasing aspect ratio variation for the string. Strings with an aspect ratio outside the aspect ratio range will not be returned as results. Note that this control type is only available for a font-based context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.8.	c		e	f	g	a		i		j	k	l	m	n	b
0.5 <= *Value* <= 1.0	Specifies the factor that determines the minimum aspect ratio of the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_CERTAINTY	Sets the certainty level for the string score. If both the string score and the string target score of a string candidate are above their respective certainty levels, the candidate is immediately considered a string result, without reading the rest of the target image for a string with higher scores (provided the specified number of strings to read has been read). Scores can be retrieved with MstrGetResult(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 70.0%.	c		e	f	g	a		i		j	k	l	m	n	b

0.0 <= *Value* <= 100.0	Specifies the certainty level for the string score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SCALE	Sets the nominal scale of the string. The scale of the string is the median scale in the X-direction of all the characters in the string. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.25 <= *Value* <= 4.0	Specifies the value of the scale.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SCALE_MAX_FACTOR	Sets the factor used to determine the upper limit (maximum permitted scale) of the string's scale range. This value is relative to the nominal scale (**MstrControl**() with M_STRING_SCALE). That is, *UpperLimit* = M_STRING_SCALE x M_STRING_SCALE_MAX_FACTOR. This value sets the possible increasing scale variation for the string. Strings with a scale outside the scale range will not be returned as results. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2.0.	c		e	f	g	a		i		j	k	l	m	n	b
1.0 <= *Value* <= 2.0	Specifies the factor that determines the minimum scale of the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SCALE_MIN_FACTOR	Sets the factor used to determine the lower limit (minimum permitted scale) of the string's scale range. This value is relative to the nominal scale (**MstrControl**() with M_STRING_SCALE). That is, *LowerLimit* = M_STRING_SCALE x M_STRING_SCALE_MIN_FACTOR. This value sets the possible decreasing scale variation for the string. Strings with a scale outside the scale range will not be returned as results. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.5.	c		e	f	g	a		i		j	k	l	m	n	b
0.5 <= *Value* <= 1.0	Specifies the factor that determines the minimum scale of the string.	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_SIZE_MAX	Sets the maximum string size (number of characters) of the string model. The space characters are not counted in the string size. The maximum string size must be greater than or equal to the minimum string size (**MstrControl**() with M_STRING_SIZE_MIN). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies no maximum string size.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the maximum string size.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SIZE_MIN	Sets the minimum string size (number of characters) of the string model. The space characters are not counted in the string size. The minimum string size must be lower than or equal to the maximum string size (**MstrControl**() with M_STRING_SIZE_MAX). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the minimum string size.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_TARGET_ACCEPTANCE	Sets the acceptance level for the string target score. To be accepted as a string result, both the string's score and the string target score must be greater than or equal to their respective acceptance levels. The string target score is computed from the unread characters that are in the region of the string in the target image. These otherwise readable characters remain unread due to user-specified constraints, such as scale and grammar restrictions. The more unread characters there are, the lower the target score will be. Scores can be retrieved with MstrGetResult(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50.0%.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies an acceptable string target score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_TARGET_CERTAINTY	Sets the certainty level for the string target score. If both the string score and the string target score of a string candidate are above their respective certainty levels, the candidate is immediately considered a string result, without reading the rest of the target image for a string with higher scores (provided the specified number of strings to read has been read). Scores can be retrieved with MstrGetResult(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 70.0%.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the certainty level for the string target score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_USER_LABEL	Sets a unique user-defined label for the specified string model. This label can be used as a means of identifying your string model, independently from its index, in the String Reader context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NO_LABEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_LABEL	Specifies that no user label is associated with the string model.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the user label of the string model. The value must be an integer that is not associated as a label with any other string model in the String Reader context. Since the same label cannot be associated with multiple string models, you cannot pass a user label if you use **MstrControl**() with M_STRING_INDEX() set to M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b

The possible ControlType and corresponding ControlValue parameter settings for a specific font (M_FONT_INDEX()) are:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a specific font																	
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_DRAW_BOX_MARGIN_X	Sets the margin in the X-direction, between the character box and the drawing box. The character box is the bounding box of the character. The drawing box is the box used for drawing purposes. For more information, see MstrDraw(). INQ															

	M_DEFAULT	Specifies the default value; the default value is 3.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the horizontal margin, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BOX_MARGIN_Y		Sets the margin in the Y-direction, between the character box and the drawing box. The character box is the bounding box of the character. The drawing box is the box used for drawing purposes. For more information, see MstrDraw(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 3.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the vertical margin, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_USER_LABEL		Sets a unique user-defined label for the specified font. This label can be used as a means of identifying your font, independently from its index, in the String Reader context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NO_LABEL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_LABEL	Specifies that no user label is associated with the font.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the user label of the font. The value must be an integer that is not associated as a label with any other font in the String Reader context. Since the same label cannot be associated with multiple fonts, you cannot pass a user label if you use **MstrControl**() with M_FONT_INDEX() set to M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings can be specified for either a font-based context or for a fontless context.

Note that the Index parameter must be set to M_FONT_INDEX() for a font-based context or to M_CONTEXT for a fontless context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a font-based or a fontless context																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																

	M_SPACE_WIDTH	Sets the width of the space character of the font. Note that the space character is not like other characters in the font. For example, you cannot set it as a constraint (MstrSetConstraint()), and it is not counted in the string size (**MstrControl**() with M_STRING_SIZE_MIN and M_STRING_SIZE_MAX). For more information, see MstrGetResult() with M_STRING_SIZE, M_STRING, and M_FORMATTED_STRING. Together with the maximum number of consecutive space characters allowed in the string (**MstrControl**() with M_SPACE_MAX_CONSECUTIVE), M_SPACE_WIDTH determines the maximum distance between 2 adjacent characters of the same string. If the distance between two characters is greater than the maximum distance, then they are considered to be part of 2 different strings. M_SPACE_WIDTH also determines how to convert the distance into space characters, when getting formatted string results (MstrGetResult() with M_FORMATTED_STRING). INQ														
	M_DEFAULT	Same as M_MEAN_CHAR_WIDTH.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies that the width of the space character is infinite. In this case, all characters on the same line are always part of the same string. Also, there will never be a space character in the read string since the distance can never be large enough to constitute a space character.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAX_CHAR_WIDTH	Specifies that the width of the space character is equal to the maximum character X-size of the font.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEAN_CHAR_WIDTH	Specifies that the width of the space character is equal to the average character X-size of the font.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_CHAR_WIDTH	Specifies that the width of the space character is equal to the minimum character X-size of the font.	c		e	f	g	a		i		j	k	l	m	n	b
	M_QUARTER_MAX_CHAR_WIDTH	Specifies that the width of the space character is equal to a quarter of the maximum character width of the font. M_QUARTER_MAX_CHAR_WIDTH is the closest to the typical space width of a standard font.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the width of the space character, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control settings of the result buffer.

Note that the Index parameter must be set to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the result buffer																	
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																	
M_RESULT_OUTPUT_UNITS	Sets whether to return results in pixels or world units. This essentially sets the output coordinate system to use. The setting of this control type will only affect functions within this module which return positional results. This control type can be changed at any time to return results in the required output units. INQ																
M_DEFAULT	Same as M_ACCORDING_TO_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. If world units are specified, calling MstrGetResult() generates an error if the result was not calculated on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler under 64-bit

```
void MstrControlInt64 (AIL_ID ContextOrResultId, AIL_INT Index, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

ContextOrResultId

See ContextOrResultId of the main function for a description.

Index

See Index of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void MstrControlDouble (AIL_ID ContextOrResultId, AIL_INT Index, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

ContextOrResultId

See ContextOrResultId of the main function for a description.

Index

See Index of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailstr11.lib.
DLL	Requires ail11.dll; ailstr11.dll.

MstrDraw

Synopsis

Draw specific features of the String Reader context or String Reader results.

Syntax

```
void MstrDraw(
    AIL_ID ContextGraId,           //in
    AIL_ID ContextOrResultStrId,   //in
    AIL_ID DstImageBufOrListGraId, //out
    AIL_INT64 Operation,           //in
    AIL_INT Index,                 //in
    const void *CharListPtr,       //in
    AIL_INT64 ControlFlag          //in
)
```

Description

This function draws specific features of the String Reader context or String Reader results in the destination image buffer or specified 2D graphics list.

After each drawing operation, the optimal image size that was needed for that operation is recorded in the String Reader context or result buffer (depending on what is passed) and can be inquired using MstrInquire with M_DRAW_LAST_SIZE_X and M_DRAW_LAST_SIZE_Y. When **M_NULL** is passed as the destination image (DstImageBufOrListGraId parameter), this function only records the optional image size for the specified drawing operation; that is, the drawing operation is not performed. This can be used to get the optimal image buffer size for the drawing operation.

Parameters

ContextGraId

Specifies the identifier of the 2D graphics context to use when drawing. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 2D graphics context																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the default 2D graphics context of the current Aurora Imaging Library application is used. > Note that there is a different default 2D graphics context for each thread.	c		e	f	g	a		i		j	k	l	m	n	b
2D graphics context identifier	Specifies a valid 2D graphics context identifier, which you have allocated using MgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextOrResultStrId

Specifies the String Reader context or String Reader result buffer from which to extract the features to draw. The String Reader context or result buffer must have been previously allocated on the system using MstrAlloc() or MstrAllocResult(), respectively.

DstImageBufOrListGraId

Specifies the identifier of the destination image buffer or 2D graphics list in which to draw. The buffer can be any valid 1- or 3-band image buffer allocated using MbufAlloc...(). The 2D graphics list must be previously allocated using MgraAllocList(). By drawing into a display's overlay buffer or associating the 2D graphics list with the display, you can also annotate an image non-destructively.

When ContextOrResultStrId is set to the identifier of a String Reader context and the Operation parameter is set to M_DRAW_CHAR, this parameter can be set to any valid Aurora Imaging Library image buffer, a 2D graphics list, or **M_NULL**. In any other case, this parameter can be set to the identifier of an image buffer or a 2D graphics list.

If specifying an image buffer, it must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

> When this parameter is set to **M_NULL**, this function only records the optional image size for the specified drawing operation; that is, the drawing operation is not performed. Use MstrInquire() with M_DRAW_LAST_... to retrieve the size.

Operation

Specifies the type of operation to perform.

The following Operation parameter value can only be set for a String Reader context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a font-based context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_CHAR	Draws a character representation of the font in the destination image. Characters are drawn from left to right, and from top to bottom. Characters that fall outside the destination image are clipped. The characters are drawn according to their draw box margin, set using MstrControl() with M_DRAW_BOX_MARGIN_X and M_DRAW_BOX_MARGIN_Y. This margin will determine the amount of white space between each character that is drawn. Note that this operation is only available if using a font-based context. The 2D graphics context cannot have its input units set to world units (MgraControl() with M_INPUT_UNITS set to M_WORLD) when drawing with this operation in either an image buffer or a 2D graphics list. If drawing in a zoomed and offset region, only integer values must be used (MgraControl() with M_DRAW_OFFSET_X, M_DRAW_OFFSET_Y, M_DRAW_ZOOM_X, and M_DRAW_ZOOM_Y set to integer values).	c		e	f	g	a		i		j	k	l	m	n	b

The following Operation parameter values can only be set for a String Reader result buffer. Note that these values can be added together to draw multiple features at once.

Unless otherwise specified, you can use these operations if results were obtained using either a font-based or a fontless context.

When applicable, features are always drawn at the location read in the target with the correct angle, scale, and aspect ratio.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a result buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DRAW_AIL_FONT_STRING	Draws all the characters of the String Reader result(s) under the bottom-left corner of the string bounding box (M_DRAW_STRING_BOX). The font associated with the 2D graphics context is used to draw the characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_STRING	Draws all the characters of String Reader result(s). Note that this operation is only available if the results were obtained using a font-based context. The 2D graphics context cannot have its input units set to world units (MgraControl() with M_INPUT_UNITS set to M_WORLD) when drawing with this operation in either an image buffer or a 2D graphics list. If drawing in a zoomed and offset region, only integer values must be used (MgraControl() with M_DRAW_OFFSET_X, M_DRAW_OFFSET_Y, M_DRAW_ZOOM_X, and M_DRAW_ZOOM_Y set to integer values).	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_STRING_BOX	Draws a box around the string that is read in the target. Note that the bounding box of the string is the bounding box of all the string's characters, including the draw margin (set with M_DRAW_BOX_MARGIN_X and M_DRAW_BOX_MARGIN_Y in MstrControl()).	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_STRING_CHAR_BOX	Draws all the character boxes of the String Reader result(s). The character box drawn is the bounding box of the character plus the draw margin (set with M_DRAW_BOX_MARGIN_X and M_DRAW_BOX_MARGIN_Y in MstrControl()).	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_STRING_CHAR_POSITION	Draws a cross at the center of the bounding box of each string's character. The character's angle, scale, and aspect ratio are taken into account.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_STRING_CONTOUR	Draws the contour of all the characters of the result(s) of the string(s) read. Note that this operation is only available if the results were obtained using a font-based context. The 2D graphics context cannot have its input units set to world units (MgraControl() with M_INPUT_UNITS set to M_WORLD) when drawing with this operation in either an image buffer or a 2D graphics list. If drawing in a zoomed and offset region, only integer values must be used (MgraControl() with M_DRAW_OFFSET_X, M_DRAW_OFFSET_Y, M_DRAW_ZOOM_X, and M_DRAW_ZOOM_Y set to integer values).	c		e	f	g	a		i		j	k	l	m	n	b

Index

Specifies the index of the font feature or result feature to draw.

Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the font or result																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Draws the default features for either fonts or results. For a String Reader context, the default value is the same as M_FONT_INDEX(). For a String Reader result, the default value is the same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_INDEX(AIL_INT FontIndex)	Draws the features of a specific font. Note that this operation is only available if using a font-based context.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	FontIndex This parameter specifies the index of the font. Set this parameter to the following:															
	M_ALL — Specifies all fonts.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* < M_NUMBER_OF_FONTS — Specifies a specific font.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Draws the features of all results.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Draws the features of a specific result.	c		e	f	g	a		i		j	k	l	m	n	b

CharListPtr
Accepts the following address:

- Data type: array of type char [optionally, in C++: a reference to a constant std::vector<char>]
  Required array size:()
- Data type: array of type short [optionally, in C++: a reference to a constant std::vector<short>]
  Required array size:()
- Data type: Set this parameter to M_NULL

Specifies an explicit list of valid characters to draw, at the specified position. This is an optional, null-terminated string.

> This parameter is not needed when drawing results and should be set to **M_NULL**.

Characters will be drawn in the order given in the CharListPtr parameter. To draw the characters on multiple lines, a space character must be inserted to separate the strings included in the null-terminated terminated string.

The array of strings must be of the right type for the encoding scheme selected (MstrControl() with M_ENCODING). At preprocessing time, each character in an explicit character list must exist in at least one of the fonts of the constraint.

ControlFlag

Specifies where in the destination image buffer to draw.

For a String Reader result, this parameter must be set to M_DEFAULT.

For a String Reader context, set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying where to draw																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Draws characters from the center of the top-left pixel of the destination image buffer; or, if drawing in a 2D graphics list, from the center of the top-left pixel of the image used at the time of annotation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ORIGINAL	Draws characters at the same offset as their original position in the definition image, for user-defined characters (set using MstrEditFont() with M_USER_DEFINED). Note that characters that are not user-defined (for example, TrueType) will be drawn at an offset of 0. > To ascertain where user-defined characters are located in the definition image, you must draw the user-defined characters in an overlay on top of the definition image. That is to say, you must draw the characters on top of their original location. For more information, refer to M_DEFINITION_OFFSET_X and M_DEFINITION_OFFSET_Y in MstrInquire().	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailstr11.lib.
DLL	Requires ail11.dll; ailstr11.dll.

MstrEditFont

Synopsis

Edit a specified font.

Syntax

```
void MstrEditFont(
    AIL_ID ContextId,        //out
    AIL_INT FontIndex,       //in
    AIL_INT64 Operation,     //in
    AIL_INT64 OperationMode, //in
    AIL_INT Param1,          //in
    const void *Param2Ptr,   //in
    const void *Param3Ptr    //in
)
```

Description

This function allows you to edit a specified font. For example, you can add or remove characters from a font, as well as normalize a character in a font. Use MstrControl() with M_FONT_ADD to add a font to the context.

Note that this function is only available for a font-based context.

Parameters

ContextId

Specifies the String Reader context that contains the font to edit. The String Reader context must have been previously allocated on the required system using MstrAlloc().

FontIndex

Specifies the index of the font to edit.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the font to edit																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FONT_INDEX(AIL_INT FontIndex)	Specifies the font to which to apply the edit settings.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	FontIndex This parameter specifies the index of the font to edit. You can set this parameter to the following:															

			0 <= *Value* < M_NUMBER_OF_FONTS	Specifies the index of the individual font.	c		e	f	g	a		i		j	k	l	m	n	b

Operation

Specifies the operation to be performed.

See the Parameter associations section for possible values that can be specified.

OperationMode

Specifies the mode of the operation.

See the Parameter associations section for possible values that can be specified.

Param1

Specifies a value that is dependent on the operation and mode chosen.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

Param2Ptr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type char [optionally, in C++: a reference to a constant std::vector<char>]
- array of type short [optionally, in C++: a reference to a constant std::vector<short>]
- char
- short

Specifies the address of the value that will be used to edit the font. The value is dependent on the operation and mode chosen.

See the Parameter associations section for possible values that can be specified.

Param3Ptr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_CONST_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- M_NULL

Specifies the address of the value that will be used to edit the font. The value is dependent on the operation and mode chosen.

Set this parameter to **M_NULL** if not used.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **Operation**, **OperationMode**, **Param1**, **Param2Ptr**, and **Param3Ptr** parameters.

- For performing the operation

To specify the operation to perform, set the Operation, OperationMode, Param1, Param2Ptr, and Param3Ptr parameters to the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the operation																	
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
OperationMode																
Param1																
Param2Ptr																
Param3Ptr																
M_CHAR_ADD	Adds character(s) to the font.															
M_SYSTEM_FONT +	Specifies that characters from a given system font (for example, TrueType and Postscript) will be added.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the size of the characters to add. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value > 6	Specifies the size, in points.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type char [optionally, in C++: a reference to a constant std::vector<char>]* **|** *array of type short [optionally, in C++: a reference to a constant std::vector<short>]* Specifies an optional string containing all of the characters to add. Two characters in one font cannot have the same value. If you try to add a character with a value that already exists in a font, the existing character will be replaced by the newly added one, unless you use M_NO_OVERWRITE. For more information, see the combination constants tables below.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that the standard characters of the font will be added. That is, A to Z, a to z, and 0 to 9.	c		e	f	g	a		i		j	k	l	m	n	b
"*String*"	Specifies the string. This must be a null-terminated string.	c		e	f	g	a		i		j	k	l	m	n	b
Param3Ptr	Data type info: Data type: *address of an AIL_CONST_TEXT_PTR [optionally, in C++: AIL_STRING]* Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b

AIL_TEXT("*System font file name*") 1	Specifies the string containing the font's file name (for example, "TrueType" and "Postscript"). This must be a null-terminated string.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED +	Specifies that user-defined characters will be added from a given image to the specified font.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the identifier of the image containing the characters to add.	c		e	f	g	a		i		j	k	l	m	n	b
Image identifier	Specifies the image identifier. The characters to add in the image must all be approximately the same size. You cannot add a character that is smaller than 6x6 pixels. Note that the size of the characters is automatically determined from the image. The characters in the image are taken from left to right and from top to bottom and are associated with the corresponding characters given in the character list. Each character in the image must be entirely connected (except for the accentuated characters) and should not be merged with other characters or other image objects. It is best to provide a character definition image that is of good quality, and that ideally contains only the characters that you want to add to the font (though this is not mandatory). If all the characters in the string you want to add cannot be found in the image, an error is returned. Even if the operation succeeds, you should draw the characters of the font to ensure that every character is defined as expected, using MstrDraw(). The image must be of type 1-band, 8-bit, unsigned. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type char [optionally, in C++: a reference to a constant std::vector<char>]* **|** *array of type short [optionally, in C++: a reference to a constant std::vector<short>]* Specifies a string that identifies the character values to associate with the character representations in the image specified in the Param1 parameter. Unlike M_SYSTEM_FONT, this setting is not optional.	c		e	f	g	a		i		j	k	l	m	n	b

		"String"	Specifies the string. This must be a null-terminated string. Two characters in one font cannot have the same value. If you try to add a character with a value that already exists in a font, the existing character will be replaced by the newly added one, unless you use M_NO_OVERWRITE. For more information, see the combination constants tables below. For a multi-string definition image, the space character (ASCII 32) must be inserted to separate the strings included in this null-terminated string. You should separate the string when one (or more) of the following situations is encountered: the characters are not all on the same line, the contrast is clearly different between characters, there is clearly a horizontal space between the characters (for example, ABC DEF).	c		e	f	g	a		i		j	k	l	m	n	b
		Param3Ptr	Data type info: Data type: *Set this parameter to M_NULL* This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED + M_SINGLE +		Specifies that the whole image will be used to define a single user-defined character in the specified font. This allows you to define special characters, which are not necessarily connected. The image is associated with the character specified by the Param2Ptr parameter. It is best to provide a character definition image that is of good quality. Note that if the character you want to add already exists in the font, the new character will replace the old one. You cannot add a character that is smaller than 6x6 pixels. By default, user-defined characters have no baseline.	c		e	f	g	a		i		j	k	l	m	n	b
		Param1	Specifies the identifier of the image containing the characters to add. The image must be of type 1-band, 8-bit, unsigned. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
		Param2Ptr	Data type info: Data type: *address of an char* **|** *address of an short* Specifies the pointer of the character to add.	c		e	f	g	a		i		j	k	l	m	n	b
		Param3Ptr	Data type info: Data type: *Set this parameter to M_NULL* This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

M_CHAR_BASELINE	Sets the baseline of the characters in the font. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Implements the default behavior. By default, for system fonts (M_SYSTEM_FONT), the baseline will be taken from that font's file. By default, for user-defined fonts (M_USER_DEFINED), the baseline will be set to M_AUTO_COMPUTE, which automatically computes the baseline to an appropriate value. When using a font to read a string model, the baseline of all characters in the string model are expected to be aligned together within a certain degree of tolerance (set using MstrControl() with M_CHAR_MAX_BASELINE_DEVIATION), otherwise the string will not be read.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the baseline.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_COMPUTE	Specifies that the baseline will be automatically computed to an appropriate value.	c		e	f	g	a		i		j	k	l	m	n	b
M_NONE	Specifies no baseline.	c		e	f	g	a		i		j	k	l	m	n	b
-1000 <= *Value* <= 1000	Specifies the baseline value, as a percentage of the character's height. The position of the baseline in a character is defined as a percentage of the height of the character. That is, 0 refers to the bottom of character, 100 refers to the top of the character, and 50 refers to the middle of character. For example, in "ABCD", you would typically set the baseline to 0 for all characters, while in "pobq", you would typically set the baseline to 0 for "ob" and approximately 25 for "pq". Note that you can set a baseline value that falls outside the limits of the character's height (Y-size).	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type char [optionally, in C++: a reference to a constant std::vector<char>]* **|** *array of type short [optionally, in C++: a reference to a constant std::vector<short>]* Specifies an optional string containing all the characters for which you want to set a baseline.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that the baseline will be set for all the characters in the font.	c		e	f	g	a		i		j	k	l	m	n	b

"*String*"	Specifies the string. This must be a null-terminated string.	c		e	f	g	a		i		j	k	l	m	n	b
Param3Ptr	Data type info: Data type: *Set this parameter to M_NULL* This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_DELETE	Deletes characters from the font.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type char [optionally, in C++: a reference to a constant std::vector<char>]* **|** *array of type short [optionally, in C++: a reference to a constant std::vector<short>]* Specifies an optional string containing all the characters to delete.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies to delete all the characters in the font.	c		e	f	g	a		i		j	k	l	m	n	b
"*String*"	Specifies the string of characters to delete. This must be a null-terminated string.	c		e	f	g	a		i		j	k	l	m	n	b
Param3Ptr	Data type info: Data type: *Set this parameter to M_NULL* This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_NORMALIZE	Normalizes the characters of the font.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_X	Specifies that the characters of the font are normalized by taking the X-size as the reference. That is, all the specified characters will be uniformly scaled to a given X-size. Note that the characters' Y-size is adjusted to maintain the same aspect ratio.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the X-size. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 8	Specifies the size, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

Param2Ptr	Data type info: Data type: *array of type char [optionally, in C++: a reference to a constant std::vector<char>]* **|** *array of type short [optionally, in C++: a reference to a constant std::vector<short>]* Specifies an optional string containing all the characters to normalize.	c	e	f	g	a	i	j	k	l	m	n	b
M_NULL	Specifies that all the characters in the font will be normalized a given X-size.	c	e	f	g	a	i	j	k	l	m	n	b
"*String*"	Specifies the string. This must be a null-terminated string.	c	e	f	g	a	i	j	k	l	m	n	b
Param3Ptr	Data type info: Data type: *Set this parameter to M_NULL* This parameter must be set to **M_NULL**.	c	e	f	g	a	i	j	k	l	m	n	b
M_SIZE_Y	Specifies that the characters of the font are normalized by taking the Y-size as the reference. That is, all the specified characters will be uniformly scaled to a given Y-size. Note that the characters' X-size is adjusted to maintain the same aspect ratio.	c	e	f	g	a	i	j	k	l	m	n	b
Param1	Specifies the Y-size. INQ	c	e	f	g	a	i	j	k	l	m	n	b
Value >= 8	Specifies the size, in pixels.	c	e	f	g	a	i	j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type char [optionally, in C++: a reference to a constant std::vector<char>]* **|** *array of type short [optionally, in C++: a reference to a constant std::vector<short>]* Specifies an optional string containing all the characters to normalize.	c	e	f	g	a	i	j	k	l	m	n	b
M_NULL	Specifies that all the characters in the font will be normalized a given Y-size.	c	e	f	g	a	i	j	k	l	m	n	b
"*String*"	Specifies the string. This must be a null-terminated string.	c	e	f	g	a	i	j	k	l	m	n	b
Param3Ptr	Data type info: Data type: *Set this parameter to M_NULL* This parameter must be set to **M_NULL**.	c	e	f	g	a	i	j	k	l	m	n	b
M_CHAR_SORT	Sorts the characters of the font.	c	e	f	g	a	i	j	k	l	m	n	b
M_ASCENDING	Specifies that the characters will be sorted, according to their character values, in ascending order.	c	e	f	g	a	i	j	k	l	m	n	b

Param1	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type char [optionally, in C++: a reference to a constant std::vector<char>]* **|** *array of type short [optionally, in C++: a reference to a constant std::vector<short>]* Specifies an optional string containing all the characters to sort.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that all the characters in the font will be sorted in ascending order.	c		e	f	g	a		i		j	k	l	m	n	b
"*String*"	Specifies the string. This must be a null-terminated string.	c		e	f	g	a		i		j	k	l	m	n	b
Param3Ptr	Data type info: Data type: *Set this parameter to M_NULL* This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_DESCENDING	Specifies that the characters will be sorted, according to their character values, in descending order.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type char [optionally, in C++: a reference to a constant std::vector<char>]* **|** *array of type short [optionally, in C++: a reference to a constant std::vector<short>]* Specifies an optional string containing all the characters to sort.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that all the characters in the font will be sorted in descending order.	c		e	f	g	a		i		j	k	l	m	n	b
"*String*"	Specifies the string. This must be a null-terminated string.	c		e	f	g	a		i		j	k	l	m	n	b
Param3Ptr	Data type info: Data type: *Set this parameter to M_NULL* This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_TYPE	Sets the type of the characters in the font.	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the characters' type. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_COMPUTE	Specifies that the characters' type will be computed automatically. The type will be set to either M_REGULAR or M_PUNCTUATION, based on the character's shape and numerical code.	c		e	f	g	a		i		j	k	l	m	n	b
M_PUNCTUATION	Specifies punctuation type characters. Punctuation characters can typically be categorized as characters that are neither letters nor numbers, such as the hyphen ('-'). Note that to be considered part of the string, a punctuation character must fall within the range of at least one regular character's Y-size.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGULAR	Specifies regular type characters. Regular characters can typically be categorized as letters and numbers. Note that a string is formed by a linear sequence of regular characters.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type char [optionally, in C++: a reference to a constant std::vector<char>]* **|** *array of type short [optionally, in C++: a reference to a constant std::vector<short>]* Specifies an optional string containing all the characters for which to set the type.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that the type will be set for all the characters in the font.	c		e	f	g	a		i		j	k	l	m	n	b
"*String*"	Specifies the string. This must be a null-terminated string.	c		e	f	g	a		i		j	k	l	m	n	b
Param3Ptr	Data type info: Data type: *Set this parameter to M_NULL* This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_THICKEN_CHAR	Thickens the characters of the font. The characters of the font should always be the thickened version, so use this Operation if the font has dotted characters. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b

		Param1	Specifies the number of character thickening iterations. Each iteration enlarges the thickness of the target character of the font. This is useful for some types of fonts (for example, dotted characters). You should choose a value large enough for the intra character dots to connect.	c		e	f	g	a		i		j	k	l	m	n	b
		0 <= *Value* <= 100	Specifies the number of iterations.	c		e	f	g	a		i		j	k	l	m	n	b
		Param2Ptr	Data type info: Data type: *array of type char [optionally, in C++: a reference to a constant std::vector<char>]* **|** *array of type short [optionally, in C++: a reference to a constant std::vector<short>]* Specifies an optional string containing all the characters to normalize.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NULL	Specifies to normalize all the characters of the font.	c		e	f	g	a		i		j	k	l	m	n	b
		"*String*"	Specifies the string. This must be a null-terminated string.	c		e	f	g	a		i		j	k	l	m	n	b
		Param3Ptr	Data type info: Data type: *Set this parameter to M_NULL* This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Combination values for [when Operation=M_CHAR_ADD: M_USER_DEFINED; M_USER_DEFINED + M_SINGLE].

You can add one of the following values to the above-mentioned values to set the foreground of the characters in the definition image when adding user-defined characters to a font.

Note that when a font is defined, it has no foreground. When performing a read operation (MstrRead()), the foreground the font is read with is set using M_FOREGROUND_VALUE in MstrControl(). In this case (when adding user-defined characters to a font), you are setting the foreground for the font in the definition image.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For setting the foreground of the characters																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FOREGROUND_BLACK	Specifies that black is the foreground color, for the definition image. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_WHITE	Specifies that white is the foreground color, for the definition image.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when Operation=M_CHAR_ADD: M_SYSTEM_FONT; M_USER_DEFINED; M_USER_DEFINED + M_SINGLE].

You can add the following value to the above-mentioned values to specify that characters will not be added to the font if they already exist.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For not overwriting characters																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NO_OVERWRITE	Specifies that the characters previously added to the font will not be overwritten. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailstr11.lib.
DLL	Requires ail11.dll; ailstr11.dll.

MstrExpert

Synopsis

Makes a diagnosis of the String Reader context and reports configuration problems.

Syntax

```
void MstrExpert(
    AIL_ID ContextId,                //in
    const AIL_ID *ImageArrayPtr,     //in
    const void *TargetStringArrayPtr, //in
    AIL_INT NbImages,                //in
    AIL_ID ResultId,                 //out
    AIL_INT64 ControlFlag            //in
)
```

Description

This function analyzes the consistency of a String Reader context using a specified image and a target string. Retrieve results using MstrGetResult(); this function returns error and/or warning codes for problems found. Use MstrInquire() to retrieve the meaning associated with these codes.

When the diagnosis of the String Reader context does not cause a general error (see MstrGetResult() with M_GENERAL), the context is ready to be used to read the image provided to **MstrExpert**(). You can call **MstrExpert**() multiple times to find and correct problems in the context.

You can limit the verify operation to a region of the image buffer, using a rectangular ROI set with MbufSetRegion().

Note that this function is only available for a font-based context.

Parameters

ContextId

Specifies the identifier of the String Reader context. The String Reader context must have been previously allocated on the required system using MstrAlloc().

ImageArrayPtr
Accepts the address of one of the following:

- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>] (and a maximum array rank for C# of 1)
  Required array size:(NbImages)
- array of type char [optionally, in C++: a reference to a constant std::vector<char>]
- array of type short [optionally, in C++: a reference to a constant std::vector<short>]

Specifies the identifier of the image to analyze. Note that you cannot provide more than one image.

This image buffer must be a 1-band buffer.

This image buffer can only have a rectangular region of interest (ROI) defined using MbufSetRegion() (with or without rotation). In addition, the region must be defined in vector format from a 2D graphics list (M_VECTOR or M_VECTOR_AND_RASTER). An error will be generated if the ROI is only in raster format (M_RASTER) or a non-rectangular shape.

TargetStringArrayPtr
Accepts the following address:

- Data type: array of type char [optionally, in C++: a reference to a constant std::vector<char>]
  Required array size:(NbImages) Note: Applies when MstrControl() with M_ENCODING is set to M_ASCII.

- Data type: array of type short [optionally, in C++: a reference to a constant std::vector<short>]
  Required array size:(NbImages) Note: Applies when MstrControl() with M_ENCODING is set to M_UNICODE.

Specifies the identifier of the string to locate. This string represents the expected strings to locate in each image analyzed using MstrRead(). The string array must be of the right type for the encoding scheme selected (MstrControl() with M_ENCODING). Only one string is supported.

NbImages

Specifies the number of images to analyze. This parameter must be set to 1.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the ImageArrayPtr parameter.

ResultId

Specifies the identifier of the String Reader result buffer in which to save the results of **MstrExpert**().

ControlFlag

Specifies the type of analysis to perform with **MstrExpert**().

To specify the type of analysis to perform, set this parameter to one of the following values. Except for M_DEFAULT, the following values can be added together.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of analysis to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to report errors and warnings.	c		e	f	g	a		i		j	k	l	m	n	b
M_ERRORS	Specifies to report errors.	c		e	f	g	a		i		j	k	l	m	n	b
M_WARNINGS	Specifies to report warnings.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailstr11.lib.
DLL	Requires ail11.dll; ailstr11.dll.

MstrFree

Synopsis

Free a String Reader context or a String Reader result buffer.

Syntax

```
void MstrFree(
    AIL_ID ObjectId //in
)
```

Description

This function deletes the specified String Reader context or String Reader result buffer identifier, and releases any memory associated with it.

All String Reader contexts and all String Reader result buffers allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

ObjectId

Specifies the identifier of the String Reader context or String Reader result buffer to free. These must have been successfully allocated (with MstrAlloc() or MstrAllocResult()) prior to calling this function.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailstr11.lib.
DLL	Requires ail11.dll; ailstr11.dll.

MstrGetResult

Synopsis

Get the specified type of result(s) from a String Reader result buffer.

Syntax

```
void MstrGetResult(
    AIL_ID ResultId,      //in
    AIL_INT Index,        //in
    AIL_INT64 ResultType, //in
    void *ResultArrayPtr  //out
)
```

Description

This function retrieves the result(s) of the specified type from a String Reader result buffer. Results are only available after calling MstrRead().

When retrieving results, you will need to know the required type and size of the array in which to store the results. Typically, the default data type is *AIL_DOUBLE*. For certain results (such as, M_TEXT, M_FORMATTED_STRING, and M_STRING), the default type of the required array depends on the selected encoding scheme (MstrControl() with M_ENCODING). You can specify any data type by explicitly adding it to the result. The size of the array depends on the result type.

Unless otherwise specified, the value settings are applicable to both font based and fontless contexts.

If your target image (MstrRead()) was associated with a camera calibration context, positional and dimensional results are, by default, returned with respect to the relative coordinate system of the image. Otherwise, these results are returned in pixels, relative to the center of top-left pixel in the target image.

If your target image was associated with a camera calibration context but you want to retrieve positional and dimensional results in pixel units, use MstrControl() with the M_RESULT_OUTPUT_UNITS control type set to M_PIXEL. However, if you set M_RESULT_OUTPUT_UNITS to M_WORLD without specifying a calibrated image in which to calculate the results, **MstrGetResult**() generates an error.

Parameters

ResultId

Specifies the identifier of the String Reader result buffer from which to retrieve results.

Index

Specifies where to get results. This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying where to get results																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies that the results of all strings will be retrieved.	c		e	f	g	a		i		j	k	l	m	n	b

M_GENERAL	Specifies that the results relating to the entire String Reader context will be retrieved.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_STRING_NUMBER	Specifies the index of the string from which to get results.	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result to retrieve.

See the Parameter associations section for possible values that can be specified.

ResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- array of type char [optionally, in C++: a reference to a std::vector<char>]
- array of type short [optionally, in C++: a reference to a std::vector<short>]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_ID
- AIL_INT
- AIL_INT16
- AIL_INT32
- AIL_INT64
- AIL_TEXT_CHAR
- char

Specifies the address in which to write results.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ResultType** parameter and possible values returned to the **ResultArrayPtr** parameter.

- For retrieving general results
- For individual strings
- For individual characters of a string
- For retrieving transformation coefficient results for individual characters of a string with a font-based context
- For retrieving errors or warnings reported by the last call to MstrExpert()

When retrieving a general result (M_GENERAL), the ResultType parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving general results																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

ResultArrayPtr - Possible values returned																
M_CHAR_NUMBER +	Retrieves the total number of characters read (all strings together).															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTEXT_ID +	Retrieves the identifier of the String Reader context used by MstrRead() to obtain the results in the result buffer. Note that this identifier might not be valid if the String Reader context has been freed using MstrFree().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ENCODING +	Retrieves the type of character encoding that MstrRead() used to generate the contents of the result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ASCII	Specifies that the characters were encoded with an 8-bit ASCII standard.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNICODE	Specifies that the characters were encoded with a 16-bit Unicode standard.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_NUMBER +	Retrieves the total number of strings read.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TEXT +	Retrieves the entire text. This includes all string characters, the inserted space characters, the string separators, and the terminating null character ("\0"). Strings are ordered in the natural Latin-based reading order. A space character is inserted in the text each time the distance between two adjacent characters exceeds the space width, at the scale of the string. The inserted space character can be set using MstrControl() with M_SPACE_CHARACTER. By default, the inserted space character is a space (ASCII code 32). Note that a string separator is inserted between different strings when getting all strings simultaneously. The string separator can be set using MstrControl() with M_STRING_SEPARATOR. By default, the string separator is a new line (ASCII code 10). Refer to M_STRING to retrieve the strings without separators. Refer to M_FORMATTED_STRING to retrieve the formatted string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type char [optionally, in C++: a reference to a std::vector<char>]* **|** *array of type short [optionally, in C++: a reference to a std::vector<short>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_TEXT_ALLOC_SIZE_BYTE +	Retrieves the number of bytes required to retrieve M_TEXT result(s). This includes all string characters, the inserted space characters, the string separators, the terminating null character ("\0"), and takes the encoding bit size into account (MstrControl() with M_ENCODING). Note that the size returned is always the number of bytes, not the number of characters.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT_END +	Retrieves whether the timeout limit was reached. You can set the timeout limit using MstrControl() with M_TIMEOUT.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the timeout limit was not reached.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the timeout limit was reached.	c		e	f	g	a		i		j	k	l	m	n	b

When retrieving individual string results, the ResultType parameter can be set to one of the following values. Results can be retrieved for a specific string, or for all strings (M_ALL), unless otherwise specified. Note that strings are ordered in the natural Latin-based reading order.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For individual strings																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_CHAR_ANGLE +	Retrieves the angle of the characters in the string, in degrees. The angle is the same for all the characters in the string; different angles for individual characters within a string cannot be obtained. The characters' angle is not necessarily equal to the string's angle. The angle is measured counter-clockwise, from the positive X-axis towards the negative Y-axis of the image.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* ┃ *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_VALUE +	Retrieves the foreground value of the string read.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* ┃ *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_FORMATTED_STRING +	Retrieves the formatted string. This includes all strings characters, the inserted space characters, and the terminating null character ("\0"). A space character is inserted in the formatted string each time the distance between two adjacent characters exceeds the space width, at the scale of the string. The inserted space character can be set using MstrControl() with M_SPACE_CHARACTER. By default, the inserted space character is a space (ASCII code 32). Note that a space character is inserted between different strings when getting all strings simultaneously. Refer to M_STRING to retrieve the strings without separators. Refer to M_TEXT to retrieve the entire text.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type char [optionally, in C++: a reference to a std::vector<char>]* **|** *array of type short [optionally, in C++: a reference to a std::vector<short>]* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_SKEW_ANGLE +	Retrieves the angle of the characters' skew in the string, in degrees. The angle is the same for all the characters in the string; different angles for individual characters within a string cannot be obtained.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING +	Retrieves the null terminated string found during the read operation. The string does not contain any space characters or string separators. Refer to M_CHAR_VALUE to retrieve all characters of all strings without the terminating null character ("\0"). Refer to M_FORMATTED_STRING or M_TEXT to retrieve all strings with their space characters and/or string separators.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type char [optionally, in C++: a reference to a std::vector<char>]* **|** *array of type short [optionally, in C++: a reference to a std::vector<short>]* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ALLOC_SIZE_BYTE +	Retrieves the number of bytes required to retrieve M_STRING result(s). This includes the terminating null character ("\0"), and takes the encoding bit size into account (MstrControl() with M_ENCODING). M_STRING_ALLOC_SIZE_BYTE is typically used when retrieving all results (M_ALL), and will return an array of string size byte (1 element for each string). Note that the size is always the number of bytes, not the number of characters.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_ANGLE +	Retrieves the angle of the string, in degrees. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise, from the positive X-axis towards the negative Y-axis of the image. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ASPECT_RATIO +	Retrieves the aspect ratio of the string. The aspect ratio of the string is the median aspect ratio of all the characters in the string. Note that this control type is only available for a font-based context.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_BL_X +	Retrieves the X-position of the bottom-left corner of the bounding box of the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_BL_Y +	Retrieves the Y-position of the bottom-left corner of the bounding box of the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_BR_X +	Retrieves the X-position of the bottom-right corner of the bounding box of the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_BR_Y +	Retrieves the Y-position of the bottom-right corner of the bounding box of the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_UL_X +	Retrieves the X-position of the top-left corner of the bounding box of the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_BOX_UL_Y +	Retrieves the Y-position of the top-left corner of the bounding box of the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_UR_X +	Retrieves the X-position of the top-right corner of the bounding box of the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_UR_Y +	Retrieves the Y-position of the top-right corner of the bounding box of the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_CHAR_NUMBER +	Retrieves the number of characters in the string found during the read operation, not including the terminating null character ("\0"); this means it returns the number of characters with results. In addition, space characters are not counted in the string size.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_CHAR_SCORE_MAX +	Retrieves the maximum character score of the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_CHAR_SCORE_MIN +	Retrieves the minimum character score of the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_MODEL_INDEX +	Retrieves the index of the string model read.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_MODEL_USER_LABEL +	Retrieves the user label of the string model read.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_POSITION_X +	Retrieves the X-position of the string. The position of the string is the position of its first character. For more information, see M_CHAR_POSITION_X.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_POSITION_Y +	Retrieves the Y-position of the string. The position of the string is the position of its first character. For more information, see M_CHAR_POSITION_Y.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SCALE +	Retrieves the scale of the string. The scale of the string is the median scale in the X-direction of all the characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SCORE +	Retrieves the string score calculated during the read operation for the entire string. The string score is the average score of the characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_TARGET_SCORE +	Retrieves the string target score calculated during the read operation for the entire string. The string target score is computed from the unread characters that are in the region of the string in the target image. These otherwise readable characters remain unread due to user-specified constraints, such as scale and grammar restrictions. The more unread characters there are, the lower the target score will be.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_TEXT; M_FORMATTED_STRING; M_STRING.

You can add the following value to the above-mentioned values to get the string's length..

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For getting the string size																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																

M_STRING_SIZE +	Retrieves the length of one or more strings, including the terminating null character ("\0"). In the case of M_TEXT, it also includes string separators. This combination constant is useful to establish the required array size to retrieve the string.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

When retrieving results for individual characters of a string, the ResultType parameter can be set to one of the following values. Results can be returned for characters of a specific string or for characters of all (M_ALL) strings. However, an array of results is always returned. Note that characters in the string are ordered by their index position in the string.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For individual characters of a string																
ResultType ResultArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CHAR_ASPECT_RATIO +	Retrieves the aspect ratio of the character. The aspect ratio of the character is the ratio of its scale in the X-direction and in the Y-direction. Note that this control type is only available for a font-based context.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BASELINE_DEVIATION +	Retrieves the baseline deviation of every read character within the string. This represents the deviation of the character's baseline from the string's baseline. For non-punctuation characters, this value is a percentage of the character's height (Y-size). For punctuation characters, this value is a percentage of the height of the tallest character within the font. For more information, refer to MstrControl() with M_CHAR_MAX_BASELINE_DEVIATION and the Character's maximum baseline deviation subsection of the Degrees of freedom section of Chapter 14: String Reader. Note that if the character's baseline is set to M_NONE, its deviation will always be returned as '0'. Note that this control type is only available for a font-based context.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_BL_X +	Retrieves the X-position of the bottom-left corner of the bounding box of each character.	c		e	f	g	a		i		j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_BL_Y +	Retrieves the Y-position of the bottom-left corner of the bounding box of each character.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_BR_X +	Retrieves the X-position of the bottom-right corner of the bounding box of each character.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_BR_Y +	Retrieves the Y-position of the bottom-right corner of the bounding box of each character.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_UL_X +	Retrieves the X-position of the top-left corner of the bounding box of each character.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_UL_Y +	Retrieves the Y-position of the top-left corner of the bounding box of each character.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_UR_X +	Retrieves the X-position of the top-right corner of the bounding box of each character.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_UR_Y +	Retrieves the Y-position of the top-right corner of the bounding box of each character.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_CONSECUTIVE_SPACE +	Retrieves the number of consecutive spaces that can be inserted between this character and the following one in the string. For more information, refer to M_SPACE_WIDTH and M_SPACE_MAX_CONSECUTIVE in MstrControl().	c		e	f	g	a		i		j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_FONT +	Retrieves the index of the font of each individual character within the string(s). The index returned is that of the character's font at the moment the MstrRead() operation was performed. Note that this control type is only available for a font-based context.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_FONT_USER_LABEL +	Retrieves the user label of the font of each individual character within the string(s). Note that this control type is only available for a font-based context.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_HOMOGENEITY_SCORE +	Retrieves the homogeneity score of each individual character within the string(s). The character's homogeneity score quantifies the similarity between the character in the target and the other characters of the string in the target.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_INDEX +	Retrieves the index of the character in the font for each individual character within the string(s). The index returned is that of the index of the character in its font at the moment the MstrRead() operation was performed. Note that this control type is only available for a font-based context.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_POSITION_X +	Retrieves the X-position of the character. The position of a character is the position of its center of gravity.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_POSITION_Y +	Retrieves the Y-position of the character. The position of a character is the position of its center of gravity.	c		e	f	g	a		i		j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SCALE +	Retrieves the scale of the character. The scale of the character is its scale in the X-direction.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SCORE +	Retrieves the score of each individual character within the string(s). The character's score quantifies the similarity between the character in the target and the character in the font; it also quantifies the similarity between the character in the target and the other characters of the string in the target.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SIMILARITY_SCORE +	Retrieves the similarity score of each individual character within the string(s). The character's similarity score quantifies the similarity between the character in the target and the character in the font.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_VALUE +	Retrieves the value of each character read.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type char [optionally, in C++: a reference to a std::vector<char>]* **|** *array of type short [optionally, in C++: a reference to a std::vector<short>]*	c		e	f	g	a		i		j	k	l	m	n	b

When retrieving transformation coefficient results for individual characters of a string, the ResultType parameter can be set to one of the following values.

Transformation coefficients allow you to transform any point from the character's source image (the image from which the characters were defined) to its corresponding position in the target image (forward), or from the read position in the target image to its corresponding position in the character's source image (reverse). This is done using the following equations:

$$x_d = A x_s + B y_s + C$$

$$y_d = D x_s + E y_s + F$$

where *A*, *B*, *C*, *D*, *E*, and *F* are the transformation coefficients (forward or reverse); x_s and y_s specify the source coordinates (character image for forward transformation or target image for reverse transformation); and, x_d and y_d specify the destination coordinates (target image for forward transformation or character image for reverse transformation).

For retrieving transformation coefficient results for individual characters of a string with a font-based context																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_A_FORWARD +	Retrieves the forward transformation coefficient A.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_A_REVERSE +	Retrieves the reverse transformation coefficient A.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_B_FORWARD +	Retrieves the forward transformation coefficient B.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_B_REVERSE +	Retrieves the reverse transformation coefficient B.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_C_FORWARD +	Retrieves the forward transformation coefficient C.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_C_REVERSE +	Retrieves the reverse transformation coefficient C.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_D_FORWARD +	Retrieves the forward transformation coefficient D.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_D_REVERSE +	Retrieves the reverse transformation coefficient D.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_E_FORWARD +	Retrieves the forward transformation coefficient E.	c		e	f	g	a		i		j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_E_REVERSE +	Retrieves the reverse transformation coefficient E.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_F_FORWARD +	Retrieves the forward transformation coefficient F.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_F_REVERSE +	Retrieves the reverse transformation coefficient F.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

When retrieving errors or warnings reported by the last call to MstrExpert(), the ResultType parameter can be set to one of the following values.

To retrieve general errors or warnings (relating to the entire String Reader context), the Index parameter must be set to M_GENERAL. To retrieve errors or warnings for a specific string, or for all strings, the Index parameter must be set to the string's index, or to M_ALL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving errors or warnings reported by the last call to MstrExpert()																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_REPORT_ERRORS +	Retrieves a list of all the errors reported by the last call to MstrExpert(). To inquire the string associated with an error, use MstrInquire() with M_REPORT_STRING.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_REPORT_NUMBER_OF_ERRORS +	Retrieves the number of errors reported by the last call to MstrExpert().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_REPORT_NUMBER_OF_WARNINGS +	Retrieves the number of warnings reported by the last call to MstrExpert().	c		e	f	g	a		i		j	k	l	m	n	b

	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_REPORT_WARNINGS +		Retrieves a list of all the warnings reported by the last call to MstrExpert(). To inquire the string associated with a warning, use MstrInquire() with M_REPORT_STRING.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For individual characters of a string, For retrieving transformation coefficient results for individual characters of a string with a font-based context, For retrieving errors or warnings reported by the last call to MstrExpert(); and for the following values: M_CHAR_ANGLE; M_FOREGROUND_VALUE; M_SKEW_ANGLE; M_STRING_ALLOC_SIZE_BYTE; M_STRING_ANGLE; M_STRING_ASPECT_RATIO; M_STRING_BOX_BL_X; M_STRING_BOX_BL_Y; M_STRING_BOX_BR_X; M_STRING_BOX_BR_Y; M_STRING_BOX_UL_X; M_STRING_BOX_UL_Y; M_STRING_BOX_UR_X; M_STRING_BOX_UR_Y; M_STRING_CHAR_NUMBER; M_STRING_CHAR_SCORE_MAX; M_STRING_CHAR_SCORE_MIN; M_STRING_MODEL_INDEX; M_STRING_MODEL_USER_LABEL; M_STRING_POSITION_X; M_STRING_POSITION_Y; M_STRING_SCALE; M_STRING_SCORE; M_STRING_TARGET_SCORE; M_CHAR_VALUE.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																	
ResultType combination value		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultArrayPtr - Possible values returned																
M_NB_ELEMENTS		Retrieves the required array size (number of elements) to store the returned values.															
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for

Tables:

- For retrieving general results
- For individual strings
- For individual characters of a string
- For retrieving transformation coefficient results for individual characters of a string with a font-based context
- For retrieving errors or warnings reported by the last call to MstrExpert()

the values listed in all the parameter association tables.

You can add the following value to the above-mentioned values to determine whether a result is available.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether results are available																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

ResultArrayPtr - Possible values returned																
M_AVAILABLE	Retrieves whether the requested result type is available for retrieval.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the requested result type is not available.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the requested result type is available.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For retrieving general results, For individual strings, For individual characters of a string, For retrieving transformation coefficient results for individual characters of a string with a font-based context, For retrieving errors or warnings reported by the last call to MstrExpert(); (exceptions might exist) and for the following value: M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested results to a required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For casting the result to a required data type																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested results to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_FLOAT* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested results to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_ID* **|** *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_TYPE_AIL_INT16	Casts the requested results to an *AIL_INT16*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT16* **|** *array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT32* **|** *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT64* **|** *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_CHAR	Casts the requested results to a *char*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an char* **|** *array of type char [optionally, in C++: a reference to a std::vector<char>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_TEXT_CHAR	Casts the requested results to an *AIL_TEXT_CHAR*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_TEXT_CHAR* **|** *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailstr11.lib.
DLL	Requires ail11.dll; ailstr11.dll.

MstrInquire

Synopsis

Inquire information about a specified String Reader context, string model, result buffer, font, or an error/warning code generated by MstrExpert().

Syntax

```
AIL_INT MstrInquire(
    AIL_ID ContextOrResultId, //in
    AIL_INT Index,            //in
    AIL_INT64 InquireType,    //in
    void *UserVarPtr          //out
)
```

Description

This function inquires information about a String Reader context, the string models or fonts contained therein, or a result buffer. In addition, this function can inquire from the String Reader context the meaning of an error/warning code generated by MstrExpert() and retrieved using MstrGetResult() with M_REPORT_ERRORS or M_REPORT_WARNINGS.

Unless otherwise specified, the inquire type settings are applicable to both font-based and fontless contexts.

If the inquired setting is set to M_DEFAULT (for example, in MstrControl()), **MstrInquire**() will return M_DEFAULT. To inquire the actual default value, add M_DEFAULT to the InquireType parameter.

Parameters

ContextOrResultId

Specifies the String Reader context or result buffer about which to inquire information. The String Reader context must have been previously allocated on the required system using MstrAlloc(). The String Reader result buffer must have been previously allocated on the required system using MstrAllocResult().

Index

Specifies that information will be inquired about the String Reader context, a specific font, a specific string model, a specified error/warning code, or a result buffer. Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a String Reader context, specific font, specific string model, result buffer, or a report generated by the last call to MstrExpert().																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default. If a String Reader context is specified, same as M_CONTEXT. If a String Reader result buffer is specified, same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_INDEX(AIL_INT FontIndex)	Specifies the index of the font about which to inquire. Note that this macro is only available for a font-based context.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															

	FontIndex This parameter specifies the index of the font. You can set this parameter to the following:													
	0 <= *Value* < M_NUMBER_OF_FONTS Specifies the index of the individual font about which to inquire. To retrieve the index of a font from its user label, set the Index parameter of **MstrInquire**() to the user label and set the inquire type to M_FONT_INDEX_FROM_LABEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_INDEX(AIL_INT StringIndex)	Specifies the index of the string about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	StringIndex This parameter specifies the index of the string. You can set this parameter to the following:															
	0 <= *Value* < M_NUMBER_OF_STRING_MODELS Specifies the index of the individual string about which you want to inquire. To retrieve the index of a string from its user label, set the Index parameter of **MstrInquire**() to the user label and set the inquire type to M_STRING_INDEX_FROM_LABEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTEXT	Specifies to inquire information about a general setting of a String Reader context.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to inquire information about a general setting of a String Reader result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
Error or warning value	Specifies the error or warning code about which to inquire. Typically, these codes are generated by the last call to MstrExpert(), and retrieved using MstrGetResult() with either M_REPORT_ERRORS or M_REPORT_WARNINGS.	c		e	f	g	a		i		j	k	l	m	n	b
User label	Specifies the label of the font or string about which to inquire, assigned using MstrControl(). Set InquireType to either M_FONT_INDEX_FROM_LABEL or M_STRING_INDEX_FROM_LABEL to retrieve the font or string index associated with the user label, respectively.	c		e	f	g	a		i		j	k	l	m	n	b

InquireType

Specifies the type of setting about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- array of type char [optionally, in C++: a reference to a std::vector<char>]
- array of type short [optionally, in C++: a reference to a std::vector<short>]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_ID
- AIL_INT
- AIL_INT16
- AIL_INT32
- AIL_INT64
- AIL_TEXT_CHAR
- char

Specifies the address in which to write the requested information. Since the **MstrInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For a context or an individual string
- For a context or a specific font
- For a String reader context
- For an individual string
- For an individual string when specifying constraints
- For an individual font
- For specifying the result for MstrGetResult()
- For inquiring about the system
- For retrieving the meaning of an error or warning report value generated by MstrExpert()
- For user label inquiries

For a String Reader context (M_CONTEXT), or a specific string in the String Reader context (M_STRING_INDEX()), the InquireType parameter can be set to the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a context or an individual string																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

M_STRING_NUMBER +		Inquires either the maximum (total) number of all strings that can be read with a String Reader context (M_CONTEXT), or the maximum number of a specific string (M_STRING_INDEX()) that can be read with a String Reader context. SET														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies that all strings will be read. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the maximum number of strings that can be read. INFO	c		e	f	g	a		i		j	k	l	m	n	b

For a fontless context (M_CONTEXT), or font-based context (M_FONT_INDEX()), the InquireType parameter can be set to the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a context or a specific font																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_CHAR_VALUE +		Inquires the character value for each character in the font-based or fontless context. Unless a data type from the table For specifying the data type is explicitly added to M_CHAR_VALUE, the string array must be of the right type for the encoding scheme selected (**MstrInquire**() with M_ENCODING).															
	UserVarPtr extra info	Data type info: Data type: *array of type char [optionally, in C++: a reference to a std::vector<char>]* **|** *array of type short [optionally, in C++: a reference to a std::vector<short>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the value of the character.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_CHARS +		Inquires the number of characters in the font-based or fontless context.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the number of characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACE_WIDTH +		Inquires the width of the space character of the font or fontless context. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_MEAN_CHAR_WIDTH.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies that the width of the space character is infinite. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAX_CHAR_WIDTH	Specifies that the width of the space character is equal to the maximum character X-size of the font.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEAN_CHAR_WIDTH	Specifies that the width of the space character is equal to the average character X-size of the font.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_CHAR_WIDTH	Specifies that the width of the space character is equal to the minimum character X-size of the font.	c		e	f	g	a		i		j	k	l	m	n	b
	M_QUARTER_MAX_CHAR_WIDTH	Specifies that the width of the space character is equal to a quarter of the maximum character width of the font. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the width of the space character, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACE_WIDTH_VALUE +		Inquires the actual width value of the space character of the font or fontless context.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the actual width value of the space character, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

For a String Reader context (M_CONTEXT), the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a String reader context																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_CHAR_STATUS +		Inquires the status of each character in the context, where the status determines whether the String Reader module should search for the character in the image. Note that this inquire type is only available for a fontless context. SET SET															
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies not to search for the character when performing a read operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to search for the character when performing a read operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTEXT_TYPE +		Inquires the type of String Reader context.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FONT_BASED	Specifies a font-based context. Such contexts must be allocated, using MstrAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
	M_FONTLESS	Specifies a fontless context. Such contexts must be restored, using MstrRestore().	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_LAST_SIZE_X +		Inquires the width of the image buffer needed by the last call to MstrDraw(). This allows you to allocate a buffer of optimal size, for the required drawing operation. Note that when you call MstrDraw() for the first time, you can pass **M_NULL** as the destination image to get the size needed for the required drawing operation. For more information, see MstrDraw().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the width of the image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_LAST_SIZE_Y +		Inquires the height of the image buffer needed by the last call to MstrDraw(). This allows you to allocate a buffer of optimal size, for the required drawing operation. Note that when you call MstrDraw() for the first time, you can pass **M_NULL** as the destination image to get the size needed for the required drawing operation. For more information, see MstrDraw().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the height of the image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENCODING +		Inquires the type of character encoding used by the String Reader context. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ASCII.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ASCII	Specifies an 8-bit ASCII standard. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNICODE	Specifies a 16-bit Unicode standard. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_CHAR_SIZE_X +		Inquires the width (X-size) of the widest, enabled character in the fontless context when it has a height of M_REF_CHAR_SIZE_Y. Note that this inquire type is only available for a fontless context. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 65 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 65536	Specifies the maximum character width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_CHAR_SIZE_X +		Inquires the width (X-size) of the narrowest, enabled character in the fontless context when it has a height of M_REF_CHAR_SIZE_Y. Note that this inquire type is only available for a fontless context. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 6 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 65536	Specifies the minimum character width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_MINIMUM_CONTRAST +		Inquires the minimum contrast between a character in the target image and its background, in order for the character to be read by MstrRead(). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 15.	c		e	f	g	a		i		j	k	l	m	n	b
	1 <= *Value* <= 255	Specifies the minimum contrast.	c		e	f	g	a		i		j	k	l	m	n	b

M_MODIFICATION_COUNT +	Inquires the current value of the modification counter. The modification counter is increased by one each time settings for the context are modified. Although you cannot identify the modification counter's contents, you can compare them throughout your application to know if the context has been altered. If the modification counter has changed you can, for example, prompt the user to save before closing the application.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the current value of the modification counter.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_FONTS +	Inquires the number of fonts in the context. Note that this inquire type is only available for a font-based context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of fonts.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_STRING_MODELS +	Inquires the number of string models in the context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of string models.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESSED +	Inquires whether the String Reader context is preprocessed. This will indicate whether preprocessing (using MstrPreprocess()) is needed to prepare the context for use.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the context is not preprocessed; you should preprocess again.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the context is preprocessed; you should not preprocess again.	c		e	f	g	a		i		j	k	l	m	n	b
M_REF_CHAR_SIZE_Y +	Inquires the reference height. Note that this inquire type is only available for a fontless context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 46 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 65536	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_REF_CHAR_THICKNESS +		Inquires the stroke width of characters in the fontless context when they have a height of M_REF_CHAR_SIZE_Y. Note that this inquire type is only available for a fontless context. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 6 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 65536	Specifies the thickness, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_CHAR_ANGLE +		Inquires whether strategies specific to angular-range will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to perform calculations specific to angular-range search strategies.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to perform calculations specific to angular-range search strategies.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_SKEW_ANGLE +		Inquires whether strategies specific to angular-range skew will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to perform calculations specific to angular-range skew search strategies.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to perform calculations specific to angular-range skew search strategies.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_STRING_ANGLE +		Inquires whether to search for the string angle. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to search for the string angle.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to search for the string angle.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACE_CHARACTER +	Inquires the character value to use as a space character within the formatted text that will be read. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is numerical ASCII code 32.	c		e	f	g	a		i		j	k	l	m	n	b
M_NONE	Specifies no space character.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the space character, as its numerical code (ASCII or Unicode). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SPEED +	Inquires the search/read speed. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_MEDIUM.	c		e	f	g	a		i		j	k	l	m	n	b
M_HIGH	Specifies a high speed.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEDIUM	Specifies a medium speed.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SEPARATOR +	Inquires the character value to use as a string separator within the formatted text that will be read. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is numerical ASCII code 10.	c		e	f	g	a		i		j	k	l	m	n	b
M_NONE	Specifies no separator character.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the separator character, as its numerical code (ASCII or Unicode). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THICKEN_CHAR +	Inquires the number of character thickening iterations. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 100	Specifies the number of character thickening iterations.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_MODE +		Inquires the threshold method of the character blob extraction. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_LOCAL_WITH_RESEGMENTATION.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOCAL	Specifies a local thresholding that uses only one threshold step that is automatically computed by the module. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOCAL_WITH_RESEGMENTATION	Specifies a local thresholding that uses multiple threshold steps and split and merge techniques. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies a global thresholding that uses the threshold value set with M_THRESHOLD_VALUE. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_VALUE +		Inquires the global threshold value when M_THRESHOLD_MODE is set to M_USER_DEFINED (otherwise it is ignored). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO_COMPUTE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO_COMPUTE	Specifies to automatically compute the global threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 255	Specifies the threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT +		Inquires the maximum read time for MstrRead(), in msec. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 2000.0 msec.	c		e	f	g	a		i		j	k	l	m	n	b

M_DISABLE	Specifies an infinite amount of read time.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum read time, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

For a specific string (M_STRING_INDEX()) in the String Reader context, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an individual string																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_CHAR_ACCEPTANCE +	Inquires the acceptance level for the character score. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50.0 %.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies an acceptable score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_ASPECT_RATIO_MAX_FACTOR +	Inquires the factor used to determine the upper limit of the aspect ratio for the characters in the string. Note that this inquire type is only available for a font-based context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.1.	c		e	f	g	a		i		j	k	l	m	n	b
1.0 <= *Value* <= 2.0	Specifies the factor that determines the maximum aspect ratio of the characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_ASPECT_RATIO_MIN_FACTOR +	Inquires the factor used to determine the lower limit of the aspect ratio for the characters in the string. Note that this inquire type is only available for a font-based context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.9.	c		e	f	g	a		i		j	k	l	m	n	b

0.5 <= *Value* <= 1.0	Specifies the factor that determines the minimum aspect ratio of the characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_HOMOGENEITY_ACCEPTANCE +	Inquires the acceptance level for the character's homogeneity score. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50.0 %.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies an acceptable score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_MAX_BASELINE_DEVIATION +	Inquires the maximum deviation that a character can have from the string's baseline. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 100	Specifies the character's maximum baseline deviation within the string model.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SCALE_MAX_FACTOR +	Inquires the factor used to determine the upper limit (maximum permitted scale) of the scale range for the characters in the string. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.1.	c		e	f	g	a		i		j	k	l	m	n	b
1.0 <= *Value* <= 2.0	Specifies the factor that determines the maximum scale of the characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SCALE_MIN_FACTOR +	Inquires the factor used to determine the lower limit (minimum permitted scale) of the scale range for the characters in the string. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.9.	c		e	f	g	a		i		j	k	l	m	n	b

0.5 <= *Value* <= 1.0	Specifies the factor that determines the minimum scale of the characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SIMILARITY_ACCEPTANCE +	Inquires the acceptance level for the character's similarity score. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50.0 %.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies an acceptable score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_CONSTRAINT_FONT +	Inquires the font index for each default font constraint. The default constraint is applied, by default, either for all positions of the string model, if no explicit constraint has been set, or for a specific position, if M_DEFAULT has been set for a specific position. This information can also be inquired using **MstrInquire**() with M_DEFAULT_CONSTRAINT_TYPE; however, it is encoded with other constant values. M_DEFAULT_CONSTRAINT_FONT always returns a single value. Note that this inquire type is only available for a font-based context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_NUMBER_OF_FONTS	Specifies the index of the individual font from which the constraint is taken. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_CONSTRAINT_TYPE +	Inquires the default constraint type. The default constraint is applied, by default, either for all positions of the string model, if no explicit constraint has been set, or for a specific position, if M_DEFAULT has been set for a specific position. M_DEFAULT_CONSTRAINT_TYPE always returns a single value.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the default constraint type.	c		e	f	g	a		i		j	k	l	m	n	b

M_FOREGROUND_VALUE +	Inquires the foreground color of the string. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FOREGROUND_BLACK.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_BLACK	Specifies that black is the foreground color of the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_BLACK_OR_WHITE	Specifies that the foreground color of the string can be either black or white.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_WHITE	Specifies that white is the foreground color of the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_CONSTRAINTS +	Inquires the number of positions in the string that have a non-default constraint. Non-default constraints are constraints that have been set to a value other than M_DEFAULT. Constraints are added to a string using MstrSetConstraint().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of positions that have a constraint.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACE_MAX_CONSECUTIVE +	Inquires the maximum number of consecutive space characters allowed in the string. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 3.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number of consecutive space characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ACCEPTANCE +	Inquires the acceptance level for the string score. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50.0 %.	c		e	f	g	a		i		j	k	l	m	n	b

0.0 <= *Value* <= 100.0	Specifies an acceptable score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE +	Inquires the nominal angle of the string. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ACCORDING_TO_REGION.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_REGION	Specifies to set the nominal angle to the angle of the rectangular ROI (set using MbufSetRegion()) in the image. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE_DELTA_NEG +	Inquires the lower limit of the string's angular range, relative to the nominal angle (MstrControl() with M_STRING_ANGLE). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.0 °.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 10.0	Specifies the lower limit of the angular range, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE_DELTA_POS +	Inquires the upper limit of the string's angular range, relative to the nominal angle (MstrControl() with M_STRING_ANGLE). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.0 °.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 10.0	Specifies the upper limit of the angular range, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ASPECT_RATIO +	Inquires the nominal aspect ratio of the string. Note that this inquire type is only available for a font-based context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b

0.5 <= *Value* <= 2.0	Specifies the ratio of pixel width to pixel height of the target.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ASPECT_RATIO_MAX_FACTOR +	Inquires the factor used to determine the upper limit of the string's aspect ratio. This value is relative to the nominal aspect ratio (MstrControl() with M_STRING_ASPECT_RATIO). Note that this inquire type is only available for a font-based context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.25.	c		e	f	g	a		i		j	k	l	m	n	b
1.0 <= *Value* <= 2.0	Specifies the factor that determines the maximum aspect ratio of the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ASPECT_RATIO_MIN_FACTOR +	Inquires the factor used to determine the lower limit of the string's aspect ratio. This value is relative to the nominal aspect ratio (MstrControl() with M_STRING_ASPECT_RATIO). Note that this inquire type is only available for a font-based context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.8.	c		e	f	g	a		i		j	k	l	m	n	b
0.5 <= *Value* <= 1.0	Specifies the factor that determines the minimum aspect ratio of the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_CERTAINTY +	Inquires the certainty level for the string score. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 70.0 %.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the certainty level for the string score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SCALE +	Inquires the nominal scale of the string. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.25 <= *Value* <= 4.0	Specifies the value of the scale.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SCALE_MAX_FACTOR +		Inquires the factor used to determine the upper limit (maximum permitted scale) of the string's scale range. This value is relative to the nominal scale (MstrControl() with M_STRING_SCALE). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 2.0.	c		e	f	g	a		i		j	k	l	m	n	b
	1.0 <= *Value* <= 2.0	Specifies the factor that determines the minimum scale of the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SCALE_MIN_FACTOR +		Inquires the factor used to determine the lower limit (minimum permitted scale) of the string's scale range. This value is relative to the nominal scale (MstrControl() with M_STRING_SCALE). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.5.	c		e	f	g	a		i		j	k	l	m	n	b
	0.5 <= *Value* <= 1.0	Specifies the factor that determines the minimum scale of the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SIZE_MAX +		Inquires the maximum string size (number of characters) of the string model. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies no maximum string size.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the maximum string size.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SIZE_MIN +		Inquires the minimum string size (number of characters) of the string model. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the minimum string size.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_TARGET_ACCEPTANCE +		Inquires the acceptance level for the string target score. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 50.0 %.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies an acceptable string target score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_TARGET_CERTAINTY +		Inquires the certainty level for the string target score. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 70.0 %.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the certainty level for the string target score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_TYPE +		Inquires the type of the string model added to the String Reader context. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_USER_DEFINED.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies a user-defined type of string model. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_USER_LABEL +		Inquires the string model's user-defined label. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NO_LABEL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_LABEL	Specifies that no user label is associated with the string model.	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the user label of the string model. INFO	c		e	f	g	a		i		j	k	l	m	n	b

For a specific string (M_STRING_INDEX()) in the String Reader context, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an individual string when specifying constraints																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_CONSTRAINT +		Inquires the constraint string at the specified index. Unless a data type from the table For specifying the data type is explicitly added to M_CONSTRAINT, the string array type corresponds to the encoding scheme selected with M_ENCODING. The string returned is null-terminated. SET You must specify a combination value from the following table: • To get the constraint string at the specified index															
	UserVarPtr extra info	Data type info: Data type: *array of type char [optionally, in C++: a reference to a std::vector<char>]* **|** *array of type short [optionally, in C++: a reference to a std::vector<short>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the constraint string.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONSTRAINT_ALLOC_SIZE +		Inquires the allocation size needed for each non-default constraint string. The allocation size of the constraint string includes the terminating null character ("\0").	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the size needed when retrieving the specified constraint string.	c		e	f	g	a		i		j	k	l	m	n	b

M_CONSTRAINT_FONT +	Inquires the font index for each non-default constraint of the string model. This information can also be inquired using **MstrInquire**() with M_CONSTRAINT_TYPE; however, it is encoded with other constant values. You can inquire the position in the string model to which the constraint applies using **MstrInquire**() with M_CONSTRAINT_POSITION. Note that this inquire type is only available for a font-based context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_NUMBER_OF_FONTS	Specifies the index of the individual font from which the constraint is taken. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CONSTRAINT_POSITION +	Inquires the position of each non-default constraint in the string model (0, 255). The default constraint is therefore applied to all positions not returned in this array.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the position of the constraint.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONSTRAINT_TYPE +	Inquires the type of each non-default constraint of the string model (for example, M_LETTER, M_DIGIT). You can inquire the position in the string model to which this constraint applies using **MstrInquire**() with M_CONSTRAINT_POSITION. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the constraint's type.	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT_CONSTRAINT +	Inquires the default constraint string. The default constraint is applied, by default, either for all positions of the string model, if no explicit constraint has been set, or for a specific position, if M_DEFAULT has been set for a specific position. The string array must be large enough to contain M_DEFAULT_CONSTRAINT + M_STRING_SIZE characters. Unless a data type from the table For specifying the data type is explicitly added to M_DEFAULT_CONSTRAINT, the string array type corresponds to the encoding scheme selected with M_ENCODING. The string returned is null-terminated.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type char [optionally, in C++: a reference to a std::vector<char>]* | *array of type short [optionally, in C++: a reference to a std::vector<short>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the default constraint string.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_CONSTRAINT; M_DEFAULT_CONSTRAINT.

You can add the following value to the above-mentioned values to get the string's length.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For getting the string size																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_STRING_SIZE +	Retrieves the length of the constraint string, including the terminating null character ("\0").															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_CONSTRAINT.

You must add the following value to the above-mentioned value to get the constraint string at the specified index.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_CONSTRAINT to specify the index																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

0 <= *Value* < M_NUMBER_OF_CONSTRAINTS +	Inquires the constraint string at the specified index. The index you provide is the index of the constraint in the list of non-default constraints, and not its actual position in the string model. You can inquire the position in the string model that this constraint string applies to using **MstrInquire**() with M_CONSTRAINT_POSITION.															
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

For a specific font (M_FONT_INDEX()) in the String Reader context, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an individual font																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_CHAR_BASELINE +	Inquires the setting for the baseline value for each character. To return the automatically calculated character baseline (when M_CHAR_BASELINE is set to M_AUTO_COMPUTE), use M_CHAR_BASELINE_VALUE. Note that this inquire type is only available for a font-based context. SET															
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_COMPUTE	Specifies that the baseline was automatically computed to an appropriate value.	c		e	f	g	a		i		j	k	l	m	n	b
M_NONE	Specifies no baseline.	c		e	f	g	a		i		j	k	l	m	n	b
-1000 <= *Value* <= 1000	Specifies the baseline value, as a percentage of the character's height. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BASELINE_VALUE +	Inquires the baseline value for each character. Note that when M_CHAR_BASELINE is set to M_AUTO_COMPUTE, this value will return the calculated character baseline. Note that this inquire type is only available for a font-based context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

	M_NONE	Specifies no baseline.	c		e	f	g	a		i		j	k	l	m	n	b
	-1000 <= *Value* <= 1000	Specifies the baseline value, as a percentage of the character's height. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SIZE_X +		Inquires the X-size of each character in the font. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-size of the character, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SIZE_Y +		Inquires the Y-size of each character in the font. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-size of the character, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_THICKNESS +		Inquires the average thickness of each character in the font. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the thickness of the character, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_TYPE +		Inquires the type of each character in the font. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO_COMPUTE	Specifies that the characters' type was computed automatically. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PUNCTUATION	Specifies punctuation type characters. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_REGULAR	Specifies regular type characters. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_TYPE_VALUE +		Inquires the type value of each character in the font. When the value of M_CHAR_TYPE is M_AUTO_COMPUTE, M_CHAR_TYPE_VALUE contains the auto-computed value (M_REGULAR or M_PUNCTUATION). For more information, see MstrEditFont() with M_CHAR_TYPE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

	M_PUNCTUATION	Specifies punctuation type characters. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_REGULAR	Specifies regular type characters. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFINITION_OFFSET_X +		Inquires the X-offset for each character in the font. The X- and Y-offsets define where the character is found in the definition image. This inquire type is only valid for user-defined characters (MstrEditFont() with M_USER_DEFINED).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NONE	Specifies that the font in the String Reader context is a system font (MstrEditFont() with M_SYSTEM_FONT).	c		e	f	g	a		i		j	k	l	m	n	b
	X-offset value	Specifies the X-offset of the character, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFINITION_OFFSET_Y +		Inquires the Y-offset for each character in the font. The X- and Y-offsets define where the character is found in the definition image. This inquire type is only valid for user-defined characters (MstrEditFont() with M_USER_DEFINED).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NONE	Specifies that the font in the String Reader context is a system font (MstrEditFont() with M_SYSTEM_FONT).	c		e	f	g	a		i		j	k	l	m	n	b
	Y-offset value	Specifies the Y-offset of the character, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BOX_MARGIN_X +		Inquires the margin in the X-direction, between the character box and the drawing box. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 3.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the horizontal margin, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BOX_MARGIN_Y +		Inquires the margin in the Y-direction, between the character box and the drawing box. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 3.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the vertical margin, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_FONT_TYPE +	Inquires the type of font added. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_USER_DEFINED.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies a user-defined type of font. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_USER_LABEL +	Inquires the font's user-defined label. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NO_LABEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_LABEL	Specifies that no user label is associated with the font.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the user label of the font. INFO	c		e	f	g	a		i		j	k	l	m	n	b

For a string result buffer (M_GENERAL), the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the result for MstrGetResult()																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_RESULT_OUTPUT_UNITS +	Inquires whether results are returned in pixel or world units. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default; the default is M_ACCORDING_TO_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for

Tables:

- For a context or an individual string
- For a context or a specific font
- For a String reader context
- For an individual string
- For an individual string when specifying constraints
- For an individual font
- For specifying the result for MstrGetResult()

the values listed in all the parameter association tables except **For inquiring about the system**, **For retrieving the meaning of an error or warning report value generated by MstrExpert()**, **For user label inquiries**.

You can add the following value to the above-mentioned values to determine the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the default value of an inquire type																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_DEFAULT		Inquires the actual default value of the specified inquire type.															
	UserVarPtr extra info	Data type info: Data type: *defined elsewhere*	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about the system on which the String Reader context or result buffer has been allocated, set the InquireType parameter to the value below.

Note that the Index parameter must be set to M_CONTEXT for a String Reader context or to M_GENERAL for a result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the system																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_OWNER_SYSTEM +		Inquires the identifier of the system on which the String Reader context or result buffer was allocated. SET SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
	System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

For retrieving the meaning of an error or warning code generated by MstrExpert(), and retrieved using MstrGetResult() with M_REPORT_..., the InquireType parameter can be set to the following value.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the meaning of an error or warning report value generated by MstrExpert()			(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
InquireType		Description															
	UserVarPtr - Possible values returned																
M_REPORT_STRING +		Inquires the error message associated with an error or warning code generated by MstrExpert().															
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For an individual string when specifying constraints, For an individual font; (exceptions might exist) and for the following values: M_CHAR_VALUE; M_CHAR_STATUS; M_CONSTRAINT_ALLOC_SIZE; 0 <= Value < M_NUMBER_OF_CONSTRAINTS; M_DEFINITION_OFFSET_X; M_DEFINITION_OFFSET_Y.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values			(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
InquireType combination value		Description															
	UserVarPtr - Possible values returned																
M_NB_ELEMENTS		Retrieves the required array size (number of elements) to store the returned values.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For a context or an individual string
- For a context or a specific font
- For a String reader context
- For an individual string
- For an individual string when specifying constraints
- For an individual font
- For retrieving the meaning of an error or warning report value generated by MstrExpert()

the values listed in all the parameter association tables except **For specifying the result for MstrGetResult**, **For inquiring about the system**, **For user label inquiries**.

You can add the following value to the above-mentioned values to determine whether an inquire type is supported.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring whether an inquire type is supported			(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
InquireType		Description															
	UserVarPtr - Possible values returned																

M_SUPPORTED		Inquires whether the specified inquire type is supported for the String Reader context.														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

For user label inquiries, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For user label inquiries																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_FONT_INDEX_FROM_LABEL +		Inquires the font index associated with a font user label if the user label is used. Note that this inquire type is only available for a font-based context.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the user label is invalid. This is returned if the specified user label is not associated with a font.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the font index associated with the specified user label.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_INDEX_FROM_LABEL +		Inquires the string model index associated with a string model user label if the user label is used.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the user label is invalid. This is returned if the specified user label is not associated with a string model.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the string model index associated with the specified user label.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For a context or an individual string, For a context or a specific font, For a String reader context, For an individual string, For an individual string when specifying constraints, For an individual font, For specifying the result for MstrGetResult(), For inquiring about the system, For user label inquiries; (exceptions might exist) and for the following value: M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested results to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination

> table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested results to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_FLOAT* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested results to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID* **|** *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT16	Casts the requested results to an *AIL_INT16*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT16* **|** *array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32* **|** *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64* **|** *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_CHAR	Casts the requested results to a *char*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an char* **|** *array of type char [optionally, in C++: a reference to a std::vector<char>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_TYPE_TEXT_CHAR	Casts the requested results to an *AIL_TEXT_CHAR*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_TEXT_CHAR* **|** *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailstr11.lib.
DLL	Requires ail11.dll; ailstr11.dll.

MstrPreprocess

Synopsis

Preprocess a String Reader context.

Syntax

```
void MstrPreprocess(
    AIL_ID ContextId,     //in
    AIL_INT64 ControlFlag //in
)
```

Description

This function prepares the specified String Reader context, font, and string model to be read.

This function must be called before the first call to MstrRead(). Changes to control settings, the String Reader context, one of its font or string models often require you to re-preprocess the context. To inquire if you need to preprocess or re-preprocess the context, use MstrInquire() with M_PREPROCESSED.

In a typical application, **MstrPreprocess**() is called once. To preprocess a String Reader context without error, the context must have at least one font, one character, and one string model.

When you save a String Reader context, the preprocessing changes are not saved. Upon restoration, you must preprocess the context again.

Parameters

ContextId

Specifies the String Reader context to preprocess. The String Reader context must have been previously allocated on the system using MstrAlloc().

ControlFlag

Specifies whether to preprocess the String Reader context. Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to preprocess the context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Preprocesses the String Reader context.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESET	Un-preprocesses the String Reader context. Un-preprocessing the String Reader context can be useful if you want to conserve system memory within an application and preserve String Reader context settings.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailstr11.lib.
DLL	Requires ail11.dll; ailstr11.dll.

MstrRead

Synopsis

Read strings from a target image.

Syntax

```
void MstrRead(
    AIL_ID ContextId,     //in
    AIL_ID TargetImageId, //in
    AIL_ID ResultId       //out
)
```

Description

This function reads one or more strings from the specified target image using the specified String Reader context. All specifications, such as the number of strings to read in the target image, the speed and/or robustness settings, the constraints on the characters in the string, and the font used, depend on the settings of the String Reader context, which can be set using MstrControl(), MstrSetConstraint() and MstrEditFont(). All settings are taken into account by **MstrRead**(), and results are stored in the specified result buffer. Results can be read from the result buffer using the MstrGetResult() function.

For a string to be read, it must not only meet all of the restrictions placed on it by the String Reader context, but it must also obey the basic rules of a string. That is, it must be a linear sequence of regular characters.

The string read will be ordered in the natural Latin-based reading order. The position of a string is the position of the center of the bounding box of its first character.

You can limit the read operation to a region of the image buffer, using a rectangular ROI set with MbufSetRegion().

The target image must be 1-band 8-bit unsigned, and cannot exceed a maximum size of 65536x65536 pixels.

Note that before performing a read operation, the String Reader context must have at least one font, one character, and one string; it must also be preprocessed (MstrPreprocess()).

If your target image was associated with a camera calibration context, positional and dimensional results are, by default, returned with respect to the relative coordinate system of the image. Otherwise, these results are returned in pixels, relative to the center of top-left pixel in the target image.

If your target image was associated with a camera calibration context but you want to retrieve positional and dimensional results in pixel units, use MstrControl() with the M_RESULT_OUTPUT_UNITS control type set to M_PIXEL. However, if you set M_RESULT_OUTPUT_UNITS to M_WORLD without specifying a calibrated image in which to calculate the results, MstrGetResult() generates an error.

In the presence of distortion, some results are meaningless when converted from real-world to pixel units (such as angle and scale). For example, if a string appears warped in the target image, but the camera calibration context compensates for this, the resulting angle is meaningful in the real-world coordinate system, and meaningless in the pixel coordinate system. If complex distortions exist, correct the image before using it with **MstrRead**().

Parameters

ContextId

Specifies the String Reader context to use for the read operation. The String Reader context must have been previously allocated on the required system using MstrAlloc(), and preprocessed using MstrPreprocess().

TargetImageId

Specifies the target image in which to read the strings.

This image buffer can only have a rectangular region of interest (ROI) defined using MbufSetRegion() (with or without rotation). In addition, the region must be defined in vector format from a 2D graphics list (M_VECTOR or M_VECTOR_AND_RASTER). An error will be generated if the ROI is only in raster format (M_RASTER) or a non-rectangular shape.

ResultId

Specifies the result buffer in which to write the results of the read operation.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailstr11.lib.
DLL	Requires ail11.dll; ailstr11.dll.

MstrRestore

Synopsis

Restore a String Reader context from disk.

Syntax

```
AIL_ID MstrRestore(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SysId,                //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID *ContextIdPtr         //out
)
```

Description

This function restores a String Reader context that was previously saved to a file, using MstrSave() or MstrStream(). This function restores all of the String Reader context's settings (including font and string model settings) that were in effect when the String Reader context was saved. A restored String Reader context is not preprocessed; therefore, you must call MstrPreprocess() before performing a read with MstrRead().

To use a fontless context, you must restore one of the predefined context files distributed with Aurora Imaging Library, which are located in the installed Contexts folder (for example, "*C:\Program Files\Aurora Imaging Library\Contexts\FONTLESS_ANPR.msr*").

When the restored String Reader context is no longer required, release it using MstrFree() unless M_UNIQUE_ID was specified during restoration; if M_UNIQUE_ID was specified, the smart identifier manages the restored String Reader context's lifetime and you must not manually free it.

Parameters

FileName

Specifies the path and name of the file from which to restore the String Reader context (for example, "*C:\mydirectory\myfile*"). The function handles (internally) the opening and closing of the file.

The predefined context files, distributed with Aurora Imaging Library, are located in the installed Contexts folder (for example, "*C:\Program Files\Aurora Imaging Library\Contexts\FONTLESS_ANPR.msr*").

To open the file on the remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").

This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Open** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("FONTLESS_ANPR.msr") 1	Specifies to restore a generic String Reader context useful to read license plates.	c		e	f	g	a		i		j	k	l	m	n	b

AIL_TEXT("FONTLESS_EUROPEAN_ANPR.msr") 1	Specifies to restore a String Reader context useful to read European license plates.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("FONTLESS_MACHINE_PRINT.msr") 1	Specifies to restore a special String Reader context that reads machine printed characters in Arial, OCR-B, or other sans-serif fonts.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the system on which to restore the String Reader context.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextIdPtr

Specifies the address of the variable in which to write the String Reader context identifier or specifies the data type that the function should use to return the String Reader context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated String Reader context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

		c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated String Reader context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_STR_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the String Reader context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.															
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored String Reader context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the String Reader context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_STR_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailstr11.lib.
DLL	Requires ail11.dll; ailstr11.dll.

MstrSave

Synopsis

Save a String Reader context to a file.

Syntax

```
void MstrSave(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID ContextId,            //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function saves all the information about the previously allocated String Reader context to disk, including all of the individual font and string settings. This information can be reloaded, using MstrRestore() or MstrStream().

> When you save the String Reader context, the preprocessing changes are not saved. Upon restoration, you must preprocess the context again.

Parameters

FileName

Specifies the name and path of the file in which to save the String Reader context. It is recommended that you use the MSR file extension for easier use with other Aurora Imaging Library software products. The function internally handles the opening and closing of this file. If this file already exists, it will be overwritten.

This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Save As** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, String Reader context files have an MSR extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ContextId

Specifies the identifier of the String Reader context to save.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailstr11.lib.
DLL	Requires ail11.dll; ailstr11.dll.

MstrSetConstraint

Synopsis

Set character constraints.

Syntax

```
void MstrSetConstraint(
    AIL_ID ContextId,         //out
    AIL_INT StringIndex,      //in
    AIL_INT CharPos,          //in
    AIL_INT64 ConstraintType, //in
    const void *CharListPtr   //in
)
```

Description

This function specifies the constraint to apply to a specified character of a specified string or of all strings.

Each constraint must include at least one character existing in one of the selected fonts. For example, if the constraint is M_DIGIT then at least one of the fonts must contain a digit.

Parameters

ContextId

Specifies the String Reader context with which to associate the constraint. The String Reader context must have been previously allocated on the required system using MstrAlloc().

StringIndex

Specifies the string(s) in the String Reader context to affect.

Note that a new string can be added to the String Reader context using MstrControl() with M_STRING_ADD.

This parameter should be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string index																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_STRING_INDEX(AIL_INT StringIndex)	Specifies that the constraint will be applied to a string or to all strings.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	StringIndex This parameter specifies the index of the string. You can set this parameter to one of the following:															

			M_ALL	Specifies that the constraint will be applied to all strings.	c		e	f	g	a		i		j	k	l	m	n	b
			0 <= *Value* < M_NUMBER_OF_STRING_MODELS	Specifies the index of the individual string to which the constraint will be applied.	c		e	f	g	a		i		j	k	l	m	n	b

CharPos

Specifies the character position in the string for which to set the constraint.

A constraint can be set for any position, even a position greater than the maximum number of characters in the string. Constraints set at a position greater than the minimum number of characters in the string are ignored (when the string read does not contain a character at that position).

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the character position																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Sets the default constraint of the string model. The default constraint of the string model is used when no specific constraint is set to a given position.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 255	Sets the character position in the string to which to apply the constraint.	c		e	f	g	a		i		j	k	l	m	n	b

ConstraintType INQ

Specifies the type of constraint to set.

By default, all constraints apply to any font in the context. For example, M_LETTER + M_DIGIT means any letter and any digit of any font is acceptable. To restrict the constraint to a specific font, see the combination value below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the constraint

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Sets the constraint to the default constraint. For a specific character position (the CharPos parameter is set to an individual value), the default is the same as the default constraint of the string model. For the default character position (the CharPos parameter is set to M_DEFAULT), the default is the same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY +	Sets the constraint to any character.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIGIT +	Sets the constraint to any digit (0 to 9).	c		e	f	g	a		i		j	k	l	m	n	b
M_DIGIT + M_LETTER +	Sets the constraint to any digit (0 to 9) and any letter (a to z and A to Z).	c		e	f	g	a		i		j	k	l	m	n	b
M_LETTER +	Sets the constraint to any letter (a to z and A to Z). Note that M_LETTER does not differentiate between upper and lower cases.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_DIGIT + M_LETTER; M_LETTER.

You can add one of the following values to the above-mentioned values to set the letter case of the constraint.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the letter case of the constraint																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_LOWERCASE	Sets the constraint to any lowercase letter (a to z).	c		e	f	g	a		i		j	k	l	m	n	b
M_UPPERCASE	Sets the constraint to any uppercase letter (A to Z).	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For specifying the constraint.

You can add the following value to the above-mentioned values to set the constraint to a specific font.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For restricting the constraint																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FONT_INDEX(AIL_INT FontIndex)	Specifies that the constraint will be restricted to a specific font. Note that this value is only available for a font-based context.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															

		FontIndex INQ This parameter specifies the index of the font. You can set this parameter to one of the following:																	
			0 <= *Value* < M_NUMBER_OF_FONTS	Specifies the index of the individual font from which the constraint is taken. When setting the constraint, the font index can be the index of a font that does not currently exist. However, if the font still does not exist at preprocessing time, an error will be logged.	c		e	f	g	a		i		j	k	l	m	n	b

CharListPtr INQ
Accepts the following address:

- Data type: array of type char [optionally, in C++: a reference to a constant std::vector<char>]
  Required array size:()
- Data type: array of type short [optionally, in C++: a reference to a constant std::vector<short>]
  Required array size:()

Specifies an explicit list of valid characters, at the specified position. This is an optional, null-terminated string.

All the specified characters must be compatible with the ConstraintType parameter definition. Each character in the character list must be unique; no character repetition is allowed in the CharListPtr parameter.

The string array must be of the right type for the encoding scheme selected (MstrInquire() with M_ENCODING).

Set this parameter to **M_NULL** if you are not specifying an explicit list.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailstr11.lib.
DLL	Requires ail11.dll; ailstr11.dll.

MstrStream

Synopsis

Load, restore, or save a String Reader context from/to a file or a memory stream.

Syntax

```
void MstrStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SysId,                  //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *ContextStrIdPtr,       //in-out
    AIL_INT *SizeByteVarPtr        //out
)
```

Description

This function can load, restore, or save a String Reader context from/to a file or memory stream.

To inquire the number of bytes necessary to save a String Reader context to memory stream, you should first call this function (**MstrStream**()) with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. This function is equivalent to a file saved using MstrSave().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file. All information about the previously allocated String Reader context is saved, including all of the individual font and string settings. However, preprocessing changes are not saved.

All of the String Reader context settings that were in effect when the String Reader context was saved will be restored. A loaded or restored String Reader context is not preprocessed, therefore you must call MstrPreprocess() before performing a read with MstrRead().

Using **MstrStream**(), you can choose to save a backwards-compatible version of the String Reader context, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore String Reader contexts saved using Aurora Imaging Library version 8.0 or above. Settings that do not exist in the lower version will be filled with default values when the String Reader context is loaded or restored.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the system on which to restore the String Reader context.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform on the String Reader context.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the String Reader context. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of stream																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the String Reader context.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextStrIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the String Reader context.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the String Reader context, in bytes. If the size is not required, you can set this parameter to **M_NULL**.

> Note that the size of a String Reader context will vary depending on the Aurora Imaging Library version specified.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **ContextStrIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.																
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

MemPtrOrFileName - Possible values passed or returned																
SysId																
Version																
ContextStrIdPtr - Possible values passed or returned																
M_INQUIRE_SIZE_BYTE	Inquires the number of bytes required to save a String Reader context to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.															
MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the String Reader context must be compatible when saved. The size of a String Reader context might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
ContextStrIdPtr	Specifies the address of the variable containing the identifier of the String Reader context.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD	Loads the content of a specified file or memory stream into a previously allocated String Reader context.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to load the context.	c		e	f	g	a		i		j	k	l	m	n	b

		M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
		AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. String Reader contexts typically have an MSR file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
		MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
	ContextStrIdPtr		Specifies the address of the variable containing the identifier of the String Reader context. All previous settings are overwritten with those of the loaded context, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the context is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE			Restores a String Reader context from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtrOrFileName		Specifies the file or memory stream from which to restore the context.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b

	AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. String Reader contexts typically have an MSR file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId		Specifies the system on which to allocate the String Reader context.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
	System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextStrIdPtr		Specifies the address of the variable in which to write the identifier of the String Reader context. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE		Saves a String Reader context to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName		Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b

	AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. For easier use with other Zebra Aurora software products, when saving a string reader context to a file, use the MSR file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version		Specifies the Aurora Imaging Library version with which the String Reader context must be compatible when saved. Note that the String Reader context can only be up to one major release older than the current version.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
ContextStrIdPtr		Specifies the address of the variable containing the identifier of the String Reader context to save.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailstr11.lib.
DLL	Requires ail11.dll; ailstr11.dll.